Applying Machine Learning Techniques to Predict Software Defects

Lew Eng Jean
Faculty of Computing and Informatics (FCI)
Multimedia University (MMU)
Selangor, Malaysia
lew.eng.jean@student.mmu.edu.my

Venushini A/P Rajendran
Faculty of Computing and Informatics (FCI)
Multimedia University (MMU)
Selangor, Malaysia
venushini@mmu.edu.my

Assoc. Prof. Ts. Dr. R. Kanesaraj A/L Ramasamy
Faculty of Computing and Informatics (FCI)
Multimedia University (MMU)
Selangor, Malaysia
r.kanesaraj@mmu.edu.my

Software Defect Prediction (SDP) plays a pivotal role in increasing software reliability and reducing maintenance by identifying faulty modules before deployment. Traditional machine learning models tend to be prone to class imbalance, noisy metrics, and scalability issues. In this research, an innovative hybrid machine learning framework based on Random Forest (RF) and Support Vector Machine (SVM) augmented with a meta-classifier for improved prediction accuracy and robustness is proposed. RF is employed for ranking feature importance and feature reduction, whereas a fast linear SVM is employed for rapid classification. A Gradient Boosting meta-model is trained on the results of RF and SVM to output final predictions. NASA JM1 dataset was used for experimentation due to its complexity and size. SMOTE and Platt Scaling methods were employed to address class imbalance and improve calibration. Experimental results demonstrate that RF-SVM+ model outperforms single classifiers in terms of high F1-score (0.8364) on the minority class and a good ROC-AUC score (0.9096), but with low run time. Results reflect the efficiency of the model to handle real-world imbalanced datasets and its potential application within automated software quality assurance pipelines.

Keywords—component, formatting, style, styling, insert (*key words****)***

I. INTRODUCTION

Amidst the fast-paced advancements in technology today, the Software Defects Prediction (SDP) utilizing machine learning (ML) techniques is essential for improving software quality and ensuring timely delivery. The target of this project is to address the challenges about class imbalance in datasets and the high computational complexity in SDP. The problem of class imbalance, where defective software instances are vastly outnumbered by non-defective ones, presents a significant obstacle in achieving accurate predictions. Additionally, the computational complexity and time required to process large datasets and build predictive models can hinder efficiency.

Software development is a complex process that involves numerous stages, from requirement gathering to deployment and maintenance. Software defects, if not detected early, can lead to financial losses, security vulnerabilities, and operational failures. Traditional testing techniques are often time-consuming and resource-intensive, making automated defect prediction models an attractive alternative. ML-based approaches offer promising solutions by analyzing historical defect data and identifying patterns indicative of future defects.

The current state of SDP research reveals several persistent challenges. One of the major issues is Class Imbalance in Dataset. Many datasets used in SDP contain disproportionately more non-defective than defective instances, leading to biased predictions and unreliable models. Shuo Feng et al.[6] and Akhilesh G.V Dhavileswarapu et al. [4] highlight that class imbalance causes models to prioritize the majority class while overlooking the minority class, leading to reduced predictive accuracy.

Another critical challenge is high Computational Complexity. The large feature sets in datasets increase computational overhead, reducing the efficiency and scalability of ML models. Xin Dong et al.[7] addressed this issue by exploring parallel processing techniques and ensemble learning methods such as Bagging, Boosting, and Stacking to accelerate model training and improve overall prediction performance. Their findings confirmed that ensemble methods significantly enhance accuracy while reducing computational load, making them more viable for time-sensitive applications.

It is important to address these challenges quickly to enhance the reliability and also the efficiency of SDP models. The proposed hybrid ML model aims to mitigate these problems by integrating robust feature selection and efficient classification techniques.

The motive of this research is to enhance SDP by addressing two major challenges: class imbalance and computational complexity. The study aims to develop a hybrid ML model that leverages RF for feature selection and SVM for classification to achieve better accuracy and efficiency. By tackling these challenges, this research contributes to improving software quality, reducing maintenance costs, and facilitating more reliable defect detection processes in software engineering. Additionally, this study seeks to bring awareness to the effectiveness of hybrid approaches in contrast to traditional single-model techniques.

The objectives of this research are:

- To develop a hybrid ML model that addresses class imbalance and improves prediction accuracy.
- To reduce computational complexity and time consumption in SDP.
- To test the accuracy of the hybrid ML model.

This study seeks to address the following research questions:

1. How does a hybrid ML model effectively address class imbalance in datasets from SDP?
2. What techniques can be employed to reduce computational complexity and time consumption in SDP?
3. How accurate is the hybrid ML model in predicting software defects compared to traditional methods?

This research focuses on two critical areas, class imbalance in datasets and computational complexity. The main target is to develop a hybrid ML model that effectively addresses the issue of class imbalance while simultaneously improving prediction accuracy. By thoroughly analyzing existing SDP datasets, the research seeks to understand the extent and impact of class imbalance, and subsequently, develop strategies to manage and mitigate this problem. Ensuring a fair and balanced representation of defects and non-defects classes within the training data is a key objective. Besides tackling class imbalance, the purpose of this research is also to identify and address the computational challenges connected with traditional ML models in SDP. This involves a detailed examination of the complexity and time requirements of these models. The research proposes and evaluates various methods to reduce computational complexity and time consumption, thereby allowing faster and more efficient processing of SDP tasks. The results of this study will provide software engineering with a more efficient strategy for SDP. The hybrid ML model can help software developers and testers identify defects earlier in the development process, reducing costs and improving software quality. Additionally, the techniques employed to address class imbalance and computational complexity can be applied to other areas of ML research. In summary, this project aims to develop a hybrid ML model to predict software defects effectively. By addressing class imbalance and reducing computational complexity, the model will improve prediction accuracy and efficiency, contributing to the advancement of software engineering implementation.

II. LITERATURE REVIEW

SDP is a critical field of study in software engineering, aiming to identify defective software components before deployment of the project. ML techniques have been extensively used in SDP due to their capability to analyze large datasets and recognize patterns that signify defects. There are many types of defects in SDP and different types of ML techniques are used to predict them. This chapter discussed multiple literature studies in different approaches, focusing on key challenges such as class imbalance, computational complexity, and the integration of hybrid ML models.

A. Class Imbalance in Dataset

Shuo Feng et al. [6] states that class imbalance are issues that take place when defective software instances are vastly outnumbered by non-defective ones, which can cause prediction models to prioritize the majority class and overlooking the minority class (Akhilesh G.V Dhavileswarapu et al., 2021). Hence, causing inaccuracy and inefficiency of the model used in SDP. To improve the software quality, defects need to be identified prior to the software's release. Therefore, Various strategies have been explored to mitigate this issue.

Hao Wang et al. [3] explored the power of combining a binary version of the gray wolf optimization (bGWO) with traditional ML classification techniques. The use of binary Gray Wolf Optimizer (GWO) is central to their method, which efficiently tackles imbalance by optimizing the selection of features relevant to defect prediction. They employ oversampling techniques to balance the dataset, ensuring that the minority class is adequately represented. This significantly boost the model's capability to accurately predict defects in imbalanced datasets. Additionally, the binary GWO algorithm is designed to manage computational complexity effectively, balancing local and global search capabilities. This approach ensures efficient feature selection and model training, making the prediction process more computationally feasible while maintaining high accuracy.

In 2021, Akhilesh G.V Dhavileswarapu and colleagues emphasized the importance of using SMOTE (Synthetic Minority Over-sampling Technique) and Complexity-based Oversampling Technique (COSTE), which is part of the synthetic data generation methods [4]. These methods add occurrences to the minority class, balancing the dataset and improving the prediction model's effectiveness. The authors highlight that ML techniques, especially those incorporating oversampling, can significantly enhance SDP and contribute to higher software quality. Additionally, the study suggests that advanced approaches like SMOTE-TUNED, which combines Neural Networks (NN) and algorithms, can further refine to handle class imbalances in SDP.

Shuo Feng et al. [6] introduced COSTE to handle the class imbalance problem in SDP. COSTE produces synthetic defective instances by combining matching sets of defective occurrences with similar complexity, improving data diversity without compromising the model's ability to detect defects. The technique outperforms existing methods like SMOTE and Borderline-SMOTE, achieving better performance in terms of false alarm probability and detection probability while considering the testing effort.

Xin Dong et al.[7] states that the high cost of detecting and resolving software defects underscores the importance of predicting these issues early in the development lifecycle. This paper looks at how the performance of seven ML and Deep Learning (DL) algorithms is on four publicly available datasets from NASA and the PROMISE repository. Then it extends the research to three Ensemble Learning (EL) techniques namely Bagging, Boosting and Stacking in order to improve prediction accuracy. The performance of the models is assessed using accuracy, precision, F1-score, recall, Area Under the Curve (AUC) and G-Mean. The results show that EL methods outperformed all the others with an AUC of 99%, a G-Mean of 96% and an average F1 score of 97%. Boosting was particularly effective in time-sensitive scenarios. Common algorithms include K-Means, K-Nearest Neighbors (KNN), Naïve Bayes (NB), Decision Tree (DT), SVM, and RF, while DL approaches like Multi-Layer Perceptron (MLP), Convolutional Neural Network (CNN), Recurrent Neural Network, and Long Short-Term Memory are also considered. Unsupervised models show comparable performance to supervised ones. This study shows the successfulness of these approaches in speculating software defects.

Tarunim Sharma et al. [8] tackled class imbalance by using EL methods like Bagging, Boosting, and Stacking, and have shown improvement in defect prediction. This review critically analyzes ensemble-based techniques for SDP from 2018 to 2021, identifying areas for enhancement through robust hyperparameter optimization, feature engineering, and the development of hybrid models. By addressing issues like incomplete validation, class imbalance, and noisy data through advanced methods like Neighborhood-based Under-sampling and Metaheuristic algorithms, the output of defect prediction models can be significantly improved. The goal is

to create a more reliable hybrid ML model with enhanced performance metrics.

In 2020, Syahana Nur'Ain Saharudin and her team discussed the challenges of class imbalance in SDP. They proposed a framework utilizing ensemble methods to boost the predictive power of models trained on imbalanced data [11]. They assessed datasets, metrics, and performance measures used in constructing prediction models, identifying 31 main studies and six types of ML techniques. NASA MDP and PROMISE repositories were frequently used datasets, with NN and Bayesian Networks (BN) being the most widely used techniques. CK Metrics Suite, including Coupling Between Object Classes (CBO), Response for a Class, and Lines of Code (LOC), were the most useful metrics. Usual performance standards included AUC, precision, recall, F-Measure, and accuracy. As ML techniques show promise, challenges remain, such as redundancy, feature irrelevance, and imbalanced data. Future research should focus on robust hyperparameter optimization, feature engineering, and hybrid models to improve prediction accuracy.

Yazan Al-Smadi et al. [14] highlighted the need for transparency in predictions with their study on explainable AI techniques. Their work aimed to enhance the intelligibility of models dealing with class imbalance. They introduced SMOTE to stabilize the data by generating synthetic samples of the lower class. SMOTE helps to ensure that the model is taught on a more representative dataset, improving its ability to accurately predict defects. Additionally, feature selection using nature-inspired search algorithms like Particle Swarm Optimization (PSO), genetic algorithm, harmony algorithm, and ant colony optimization can further enhance model performance. Ant colony optimization has displayed amazing results in improving feature selection. Balancing the dataset and selecting relevant features are crucial steps in developing a robust SDP model that can reliably identify defect-prone components.

Asmaa M Ibrahim et al. [15] focused on the effect of class imbalance on model capabilities. The study employs SMOTE to balance the data by producing synthetic samples for the outnumbered class. This ensures that the model is trained on a more representative dataset, enhancing its ability to predict defects accurately. Additionally, nature-inspired search algorithms like PSO, genetic algorithm, harmony algorithm, and ant colony optimization are used for feature selection, further improving model performance. The balanced versions of ensemble learning algorithms, such as Random Under-Sampling Boost, Easy Ensemble, Balanced Bagging, and Balanced RF, show improved recall and ROC-AUC values by incorporating re-sampling strategies. However, these methods also cause a drop in precision, indicating a balance between detecting defective methods and conserving testing resources. Balanced RF demonstrates effective recall and ROC-AUC values, yet may not be practical when precision and resource conservation are critical.

Muhammad Jumare and his team [17] concentrated on ensemble learning techniques for SDP, demonstrating that their methods significantly improved the identification of defective instances. The research aims to detect the most useful ML and DL models for early defect detection in software development. The study uses RF, SVM, NB, Artificial Neural Network (ANN), and CNN algorithms on the JM1 NASA dataset with 10,885 instances and 22 code metric attributes. To address class imbalance, a hybrid sampling technique (SMOTE-Tomek) is applied. Models are assessed using accuracy, precision, recall, F1-score, and AUC-ROC. The RF model outperforms others, achieving 82.3% accuracy, 96.8% recall, 0.898 F1-score, and 83.7% precision. The CNN model also shows promise with 81.95% accuracy and 95.90% recall. These results demonstrate improved handling of class imbalance and predictive performance. High recall rates highlight the effectiveness of these models, especially RF, in identifying defects. However, balancing precision and recall remains a challenge. The findings offer refined methodologies for SDP and open avenues for further research in applying ML to software engineering challenges.

Aimen Khalid et al. [22] oversees a systematic review of various defect prediction techniques. This study examines various ML techniques, as well as corrected ML techniques, on a publicly accessible dataset to enhance model performances like accuracy and precision. Previous investigations indicate that accuracy can be further enhanced. To this end, K-means clustering was employed for categorizing class labels, followed by the application of classifying algorithms to chosen variables. PSO was implemented to boost the ML models. The capability of that trained model was evaluated by using precision, accuracy, recall, F-measure, performance error metrics, and a confusion matrix. The output shows that both the standard and corrected machine learning models performed exceptionally well; however, the SVM and corrected SVM models exceed the others with accuracies of 99% and 99.80%, respectively. The results of the rest such as NB, Optimized NB, RF, Optimized RF, and ensemble approaches are as follows: 93.90%, 93.80%, 98.70%, 99.50%, 98.80%, and 97.60%, correspondingly. Thus, the study achieved maximum accuracy compared to previous research, fulfilling its objective.

Faseeha Matloob et al. [24] reviewed the challenges of class imbalance in SDP. They proposed strategies such as cost-sensitive learning and ensemble methods to improve model performance. This study conducts a systematic literature review on the implementation of EL for SDP, exploring research papers published since 2012 in these prominent online sources: ACM, IEEE, Springer Link, and Science Direct. Multiple research questions addressing various sides of ensemble learning in SDP were formulated, and 46 relevant papers were selected through a detailed systematic research procedure. This study provides a succinct overview of the most recent trends and advancements in EL for SDP, serving as a foundation for future innovations and evaluations. The findings reveal that RF, boosting, and bagging are constantly used ensemble techniques, while stacking, voting, and Extra Trees are less commonly used. Researchers have proposed various promising frameworks, such as EMKCA, SMOTE-Ensemble, MKEL, SDAEsTSE, TLEL, and LRCR, utilizing EL techniques. Commonly used performance metrics include AUC, accuracy, F-measure, recall, precision, and MCC. WEKA emerged as a commonly implemented platform for ML. Empirical analyses indicate that feature selection and data sampling are essential pre-processing steps that enhance the capabilities of ensemble classifiers. Also, EL techniques outperform individual classifiers in SDP.

Karedla Chayadevi et al. [16] explored software defect prediction (SDP) using NB and RF classifiers to identify defect-prone modules early in the software development

cycle. Their study analyzed object-oriented metrics and benchmark datasets from the PROMISE repository, evaluating seven machine learning techniques at both method-level and class-level. Results indicated that NB performed best for class-level datasets, while RF excelled for method-level datasets. The study also highlighted key challenges in SDP, including class imbalance and dimensionality reduction, suggesting that feature selection and oversampling techniques can enhance predictive accuracy. Future work aims to integrate these techniques with ML classifiers to improve defect detection reliability.

Jyothi Kethireddy et al. [18] propose a software defect prediction model based on multiple ML algorithms, including ANN, RF, RT, DT, LR, GP, SMOreg, and M5P. The study evaluates these models on a publicly available dataset using various performance metrics such as accuracy, precision, recall, F-measure, and AUC. The results indicate that ML-based defect prediction models can effectively forecast software defects, with the best models achieving an AUC of 0.81. The authors suggest that integrating ML techniques into software development can improve defect detection and overall software quality. Future work aims to explore genetic algorithm-based prediction models and cost-benefit analyses of defect prediction techniques.

Karishma Sahni et al. [28] conducted a SLR using distance-based classification algorithms with the Mahalanobis distance function. It highlights the limitations of existing implementations, which do not consider label information (defective or defect-free) from the training data. The research aims to improve the performance of defect prediction models while acknowledging practical challenges such as class imbalance, correlated information, random errors, and noise. Additionally, the study discusses the issue of overfitting when models become excessively complex.

Finally, Xin Fan and colleagues [27] examined various resampling techniques, including SMOTE and hybrid methods. They introduced a SDP method based on stable learning (SDP-SL), which integrates code visualization techniques and residual networks. The approach converts code files into visual representations and constructs a defect prediction model from these images. Notably, target project data is not required during model training. By continuously adjusting the importance of source project samples, SDP-SL removes feature dependencies. This approach captures the data's "invariance mechanism," thereby improving defect prediction accuracy. Through comparative experiments conducted on 10 open-source projects from the PROMISE dataset, SDP-SL was found to surpass other within-project defect prediction methods. The results showed an improvement ranging from 2.11% to 44.03% in terms of the F-measure. In cross-project defect prediction, SDP-SL provided a 5.89%–25.46% improvement in prediction performance compared to other methods, effectively enhancing both within- and cross-project defect predictions.

Frequently, datasets like NASA MDP and PROMISE were used, offering a diverse range of metrics and defect labels. The evaluation metrics often included accuracy, precision, recall, and AUC, crucial for assessing model performance in imbalanced scenarios.

Table 2.1 summarizes various ML techniques used to handle class imbalance in SDP. It highlights methods such as SMOTE, COSTE, and ensemble learning, providing insights into their advantages, performance metrics, and effectiveness in balancing datasets.

TABLE I. OVERVIEW OF TECHNIQUES FOR ADDRESSING CLASS IMBALANCE

No	Techniques	Tools	Language	Dataset	Performance Metrics	Testing	Result
1	bGWO for feature selection. ML classifiers: KNN, DT, SVM, NB, ANN	Matlab	Python	NASA (CM1, JM1, KC1, KC2, PC1) include metrics like McCabe's Cyclomatic Complexity, Halstead metrics, and line counts.	Accuracy, precision, recall, and F1-score.	Feature selection was performed using the bGWO to reduce dimensionality. ML classifiers (KNN, DT, SVM) were trained and tested using the selected features. Performance metrics such as accuracy, precision, recall, and F1-score were calculated on the test sets of the NASA datasets to evaluate the defect prediction models.	Improved prediction performance with optimal feature subsets.
2	Oversampling techniques (e.g., COSTE and SMOTE) to balance datasets. ML model: DT, RF, NB, NN	Jupyter Notebooks or IDEs like PyCharm (common)	Python (+libraries like Scikit-learn and TensorFlow)	NASA, PROMISE Github, Kaggle and other public sources for additional datasets	Accuracy, recall, precision, and ROC curves.	Testing involved addressing class imbalance with oversampling techniques (e.g., SMOTE and COSTE). ML models like RF, DT, and NN were evaluated. Metrics like accuracy, recall, precision, and ROC curves were used to validate the effectiveness of the defect prediction models.	Improved class distribution and defect prediction accuracy.
3	M-SMOTE for balancing data.	TensorFlow, pytorch	Python	Open-source datasets (unspecified exact sources). most likely NASA,PROMISE,Github Modified SMOTE (M-SMOTE) was applied to balance these datasets.	Accuracy: 95.32%, Recall: 93.3%, F1-score: 91.355%, MCC: 94.98%.	Data pre-processing included normalization and balancing with M-SMOTE. Features were extracted using Focal BERT (F-BERT), which reduced dimensionality. The hybrid Bi-CGRU model was trained and tested, and hyperparameters were optimized using the Hybrid Levy Rao (HLR) optimization technique. Metrics like F1-score, accuracy, recall,	Reduced overfitting and computational cost with enhanced model performance.

						and Matthews Correlation Coefficient (MCC) were applied to assess model performance.	
4	ML Algorithms: K-Means, KNN, NB, DT, SVM, MLP, CNN. EL methods (Bagging, Boosting, Stacking).	Not specified, likely standard ML libraries.	Not specified.	4 public datasets from NASA and PROMISE repository. (Ant-1.7, Camel-1.6).	Accuracy, Precision, Recall, F1-score, AUC, G-Mean.	K-fold cross-validation (K=10) and SMOTE for data balancing.	EL achieved the highest AUC of 99%; G-Mean of 96%; average F1-score of 97%.
5	EL techniques (Bagging, Boosting).	Not specified.	Not specified.	Various datasets from 2018 to 2021.	Not specified; implied metrics include accuracy and AUC.	Not specified; likely involves cross-validation.	Ensemble methods demonstrated improved defect prediction performance.
6	Various ML techniques including NN, SVM, and Ensemble Learning.	Not specified; likely standard ML libraries.	Not specified.	31 studies reviewed, primarily using PROMISE and NASA datasets.	AUC, F-Measure, Precision, Recall, Accuracy.	Systematic literature review; no specific testing methodology mentioned.	ML techniques can predict bugs, but challenges remain in model performance.
7	11 ML classifiers: LogReg, KNN, DT, RF, SVM, AdaBoost, Gradient Boosting, Stochastic Gradient Boosting, Extreme Gradient Boosting, Categorical Boosting, Stacked Generalization. Feature selection using nature-inspired algorithms: PSO, GA, HA, ACO.	Shapley additive explanation (SHAP) for model interpretation.	Not specified.	Twelve datasets from NASA Metrics Data Program (CM1, JM1).	Accuracy, ROC-AUC.	SMOTE for data balancing, 10-fold cross-validation.	Gradient boosting and other classifiers achieved over 90% accuracy and ROC-AUC.
8	EL algorithms: AdaBoost, Gradient Boost, Bagging, RF, Random Under Sampling Boost, Easy Ensemble, Balanced Bagging, Balanced RF.	Not specified; likely standard ML libraries.	Java.	ELFF datasets from 23 open-source Java projects (Apache Commons, JFreeChart).	Recall, ROC-AUC, Precision, F1-score.	Resampling techniques to handle class imbalance, including SMOTE and random undersampling.	Balanced RF achieved the best results regarding recall (0.85) and ROC-AUC (0.92).

9	ML Algorithms: RF, SVM, NB, ANN, CNN Hybrid sampling technique (SMOTE-Tomek) to address class imbalance.	Not specified; likely standard ML libraries.	Not specified.	JM1 NASA software defect dataset, consisting of 10,885 instances with 22 code metric attributes.	Accuracy, Precision, Recall, F1-score, AUC-ROC.	Models evaluated using a 70:30 train-test split, with hybrid sampling applied to the training dataset.	RF achieved an accuracy of 82.3%, recall of 96.8%, F1-score of 0.898, and precision of 83.7%.
10	ML Techniques: SVM, NB, RF, Ensemble approaches. K-means clustering to sort class labels into categories. PSO for optimizing ML models.	Not specified; likely standard ML libraries.	Not specified.	CM1 dataset from the PROMISE repository, containing 498 instances and 22 features.	Accuracy, Precision, Recall, F-measure, performance error metrics, confusion matrix.	Models evaluated using cross-validation and performance metrics calculated on test data.	SVM (99%) and optimized SVM (99.80%) models achieved the highest accuracy.
11	Review of EL techniques: RF, Boosting, Bagging, Stacking, Voting, Extra Trees. Hybrid classifiers and frameworks proposed in various studies.	WEKA widely adopted as a platform for ML	Not specified.	Various datasets from the PROMISE repository and NASA datasets.	AUC, accuracy, F-measure, recall, precision, Matthews Correlation Coefficient (MCC).	Systematic literature review; no specific testing methodology mentioned.	Ensemble methods generally outperform individual classifiers, with significant improvements noted in various studies.
12	NB, RF, Feature Selection, Oversampling	Not mentioned	Not mentioned	PROMISE repository (benchmark datasets)	Accuracy	Evaluated at method-level and class-level	NB performed best for class-level datasets RF excelled for method-level datasets Identified key challenges: class imbalance & dimensionality reduction Suggested feature selection and oversampling for better predictive accuracy
13	ANN, RF, RT, DT, LogReg, GP, SMOreg, M5P	Not mentioned	Not mentioned	Publicly available dataset	Accuracy, Precision, Recall, F-measure, AUC	Compared multiple ML models using performance metrics	ML-based models effectively predict defects

							Best models achieved AUC of 0.81 Suggested future work: Genetic algorithm-based models & cost-benefit analysis
14	Distance-based classification using Mahalanobis Distance Function	Not mentioned	Not mentioned	Not specified	Not explicitly mentioned	Systematic Literature Review (SLR)	Existing Mahalanobis function implementations do not consider label information
15	SDP-SL method combining code visualization techniques and residual networks. Sample reweighting to eliminate dependencies between features.	Not specified; likely standard ML libraries.	Not specified.	10 open-source projects from the PROMISE dataset.	F-measure, accuracy, precision, recall.	Comparative experiments conducted on multiple projects, focusing on within-project and cross-project defect prediction.	SDP-SL outperformed other methods in both WPDP and CPDP, with improvements in F-measure ranging from 2.11% to 44.03%.

B. Computational Complexity and Time Consumption

Anurag Mishra and Ashish Sharma [5] took a different approach by implementing DL-based CI/CD SDP technique to address challenges like computational complexity, time consumption, and high-loss functions in existing methods. By utilizing the Modified Synthetic Minority Over-Sampling Technique (MSMOTE) to balance the data and Focal Bidirectional Encoder Representations from Transformers (F-BERT) for optimal feature extraction, their model enhances prediction accuracy. The Bidirectional Long Short-Term Memory (Bi-LSTM) integrated with convolutional Gated Recurrent Units (GRU) (Bi-CGRU) is optimized using HLR optimization, achieving 95.32% accuracy, 93.3% recall, 94.98% Matthews correlation coefficient, and a 91.355% F1-score. This approach results in less labeling noise and reduced wait time, making it effective for defect prediction in CI/CD pipelines.

Xin Dong et al. [7] implemented techniques of using parallel processing to accelerate model training and evaluation. In this study, the performance of seven commonly used ML and DL algorithms was evaluated on the NASA and PROMISE repository that is available online. Additionally, three classical EL methods (Bagging, Boosting, and Stacking) were applied to enhance prediction performance. The performance metrics included accuracy, precision, F1-score, recall, AUC, and G-Mean. The results demonstrated that ensemble learning significantly outperformed the other algorithms, achieving the highest AUC and G-Mean, which is 99% and 96% respectively, and an average F1-score of 97%. In high-priority scenarios, the Boosting method proved to be an effective choice due to its shorter runtime and comparable capability to the other ensemble techniques. Simulation results confirmed that EL methods significantly exceeded the capability of the seven frequently employed ML and DL algorithms, particularly under time-sensitive conditions.

Asmaa M Ibrahim et al. [15] focus to addresses the computational complexity issues in SDP by implementing efficient techniques and algorithms. Specifically, the implementation of EL algorithms like Ada-Boost, Gradient Boost, Bagging, and RF is explored, as well as their balanced versions (Random Under-Sampling Boost, Easy Ensemble, Balanced Bagging, and Balanced RF). The results highlight that while traditional ensemble algorithms often struggled with high computational complexity and imbalanced datasets, the balanced versions significantly improved recall and ROC-AUC values by incorporating re-sampling strategies. However, these improvements came at the cost of reduced precision, due to the re-sampling methods discarding instances from the majority class. The study emphasizes the need for a trade-off between computational efficiency and prediction accuracy. By optimizing feature selection using techniques like PSO, genetic algorithms, harmony algorithms, and ant colony optimization, the research demonstrates potential pathways to handle large, imbalanced datasets more effectively. The implementation of SMOTE for data balancing further contributes to managing computational complexity, leading to improved model performance and resource allocation in SDP.

Aimen Khalid et al. [22] and colleagues addresses computational complexity issues in SDP by implementing efficient techniques and algorithms. Four real datasets provided by NASA were preprocessed using the SMOTE and Label Encoding (LE) to handle data imbalances and normalization. Thirteen ML models were tested, including RF Regression, Linear Regression (LR), NB, DT Classifier, Logistic Regression (LogReg), K-Neighbors Classifier, AdaBoost, Gradient Boosting Classifier, Gradient Boosting Regression, XGBR Regressor, XGBoost Classifier, Extra Trees Classifier, and SVM. The XGBR model demonstrated the best performance considering accuracy, mean square error, and R2 score. Explainable AI techniques such as Local Interpretable Model (LIME) and SHapley Additive exPlanations (SHAP) were used to identify key software fault features, confirming that the models are precise and interpretable. Features like the Number of Static Invocations, Depth Inheritance Tree, and CBO were identified as the ultimate influential software faults. The study's methodology effectively addresses computational complexity, data imbalance, and feature selection, leading to significant improvements in SDP performance.

Manpreet Singh et al. [25] introduced a refined ML-based CPSDP model that incorporates innovative structural code metrics as input features. These metrics capture the code's structure and organization, providing deeper insights into its architecture, such as coupling, polymorphism, lack of cohesion, complexity, inheritance coupling, and abstraction, more effectively than existing metrics. The study involves preparing datasets from 18 Java projects based on these metrics and introducing a procedure based on t-tests for selecting projects to enhance the quality of training datasets. Extreme Gradient Boost (XGBoost) was selected as the ultimate ML algorithm for the proposed model, solely based on AUC, accuracy, and the newly proposed Balance Between AUC and Accuracy (BBAA) metric. Results show notable improvement in performance, with the suggested model achieving a 7% higher AUC than existing models. The model's training and prediction times are similar to existing ML-based CPSDP models and significantly better than DL-based models, making it a more lightweight and efficient solution. However, the model's performance depends a lot on the training data's standard, necessitating the selection of suitable projects, which is a time-consuming process.

In 2023, Sagheer Abbas and colleagues conducted a study that first preprocesses four real datasets offered by NASA using the SMOTE alongside with the Label Encoding techniques [24]. Later, thirteen ML models were experimented with, including RF Regression, LR Regression, Naïve Bayes (NB), DT Classifier, LogReg, KNeighbors Classifier, AdaBoost, Gradient Boosting Classifier, Gradient Boosting Regression, XGBR Regressor, XGBoost Classifier, Extra Trees Classifier, and SVM. Among them, XGBR outperformed others in terms of accuracy, mean square error, and R2 score. Explainable Artificial Intelligence techniques like Local Interpretable Model (LIME) and SHapley Additive exPlanations (SHAP) were used to determine significant software fault features. The study found that the Number of Static Invocations, Depth Inheritance Tree, and CBO were the most impactful features. LIME demonstrated the highest true positive rates for these features. This approach addresses computational complexity by ensuring balanced datasets, optimizing feature selection, and enhancing model interpretability, leading to improved SDP.

Xin Fan et al. [27] tackle computational complexity issues in defect prediction by proposing a SDP method based on stable learning (SDP-SL). The method combines code visualization techniques and residual networks, transforming

code files into images and constructing a defect prediction model from these images. During training, target project data are not required as prior knowledge. The model dynamically adjusts the weights of source project samples to eliminate feature dependencies, capturing the "invariance mechanism" within the data. Comparative experiments on 10 open-source projects in the PROMISE dataset demonstrate that SDP-SL outperforms other methods, showing improvements of 2.11%–44.03% in within-project defect prediction and 5.89%–25.46% in cross-project defect prediction. This approach addresses computational complexity by leveraging stable learning principles and eliminating feature dependencies, leading to enhanced defect prediction performance.

Misbah Ali et al. [31] introduces an intelligent ensemble-based SDP model that merges various classifying agents through a two-stage prediction process. In the first stage, four supervised ML algorithms—RF, SVM, NB, and Artificial Neural Network—are optimized to achieve high accuracy. In the second stage, these classifiers' predictive accuracies are amalgamated into a voting ensemble to produce the final predictions, further improving accuracy and reliability. The model was evaluated using seven actual defect datasets from the NASA MDP repository that is widely available online. They are CM1, JM1, MC2, MW1, PC1, PC3, and PC4, and have demonstrated exceptional accuracy, achieving better results than twenty top-tier defect prediction models. However, the effectiveness of the model is heavily influenced by the quality of training data. Disparities, missing data, or noise in the training dataset can adversely affect prediction performance. Moreover, ensemble-based models trained on historical data may struggle to adapt to unexpected changes in project dynamics or new development paradigms.

Muhammad Azam et al. [32] examines various ML algorithms for predicting software defects, using five NASA datasets (JM1, CM1, KC1, KC2, and PC1). The study employs different ML techniques, including BN, LogReg, MLP, Ruler Zero-R, J48, Lazy IBK, SVM, NN, RF, and Decision Stump, to identify the largest subset of predictable defects. LogReg demonstrated the highest accuracy at 93%, while the BN algorithm showed an average increase in accuracy by 8% using feature selection. The aim of this research is to bridge the gap between data mining and software engineering, ultimately leading to higher success rates in software development.

Table 2.2 presents techniques used to mitigate high computational costs in SDP. It includes approaches like parallel processing, feature selection, and optimized ensemble learning, showcasing their impact on model efficiency and scalability.

TABLE II. OVERVIEW OF TECHNIQUES FOR ADDRESSING COMPUTATIONAL COMPLEXITY

No	Techniques	Tools	Language	Dataset	Performance Metrics	Testing	Result
1	M-SMOTE, F-BERT for feature extraction, Bi-LSTM with Convolutional GRU, HLR optimization.	Not specified, but DL frameworks are implied.	Not specified.	Open-source CI/CD datasets with labeled numerical data.	Accuracy (95.32%), Recall (93.3%), F1-score (91.355%), MCC (94.98%).	Dataset balancing, feature extraction, and classification were tested using cross-validation and time-domain labeling.	Reduced computational complexity and waiting time, high accuracy, and efficient defect prediction.
2	Bagging, Boosting, Stacking; compared with KNN, SVM, CNN, etc.	Not specified.	Implemented in a simulated environment using Python or similar tools for ML.	NASA and PROMISE repositories (Ant-1.7, PC1, JM1).	Accuracy, Precision, F1-score (0.97 average), G-Mean (0.96), AUC (0.99).	K-fold cross-validation (K=10), runtime comparison.	Boosting methods were efficient in time-sensitive scenarios; ensemble methods outperformed individual algorithms.
3	Ensemble methods: Bagging, Boosting, Stacking, and RF. Feature selection techniques to improve efficiency.	Not specified.	Not specified.	Various datasets from the PROMISE repository.	Accuracy, Precision, Recall, F1-score.	Evaluated computational efficiency and time consumption of ensemble methods.	Ensemble methods generally required more computational resources, with training times ranging from 15 to 30 minutes depending on the dataset size.
4	ML algorithms (e.g., XGBR, RF, LogReg, etc.) Explainable AI methods (LIME, SHAP).	Not explicitly mentioned but involves standard ML libraries and XAI tools.	Likely Python or R; integration with SHAP and LIME libraries.	PROMISE datasets (PC1, Baseline, CM1, JM1).	Accuracy, MSE, R^2; feature importance analyzed with SHAP/LIME.	80/20 train-test split, compared multiple ML models, explainability via XAI techniques.	XGBR was the best-performing model; LIME provided more accurate defect feature analysis.
5	Review of ML techniques: SVM, RF, DT, and NN. Discussion on the impact of feature selection on computational efficiency.	Not specified.	Not specified.	PROMISE, NASA	Accuracy, Precision, Recall, F1-score.	Systematic review of computational efficiency across different algorithms.	SVMs and Neural Networks were noted for their high computational complexity, often requiring over 30 minutes for training, while RF and DT were more efficient, typically under 10 minutes.

6	Data fusion. Feature selection (Correlation FS, Consistency FS). Ensemble ML methods (Bagging, Voting, Stacking). Fuzzy logic for decision-level integration.	Not specified; likely standard ML frameworks.	Not specified.	Five fused datasets from NASA's repository (MW1, PC1, PC3, PC4, CM1).	Accuracy, F-measure, Matthews Correlation Coefficient (MCC), Receiver Operating Characteristic (ROC).	70-30 train-test split; testing performed using a nested ensemble approach.	Achieved 92.08% accuracy, demonstrating improved efficiency through data fusion and ensemble learning.
7	Stable Learning (Sample Reweighting). Code visualization (conversion of code to RGB images). Residual Networks (ResNet18).	ResNet18; Random Fourier Features (RFF).	Likely Python (ML and NN).	PROMISE datasets (10 open-source Java projects).	F-measure, Cross-project prediction improvement percentages.	Comparative experiments with within-project and cross-project defect prediction tasks.	Outperformed other methods by improving F-measure by 2.11–44.03% for WPDP and 5.89–25.46% for CPDP.
8	RF, SVM, NB, ANN, Voting Ensemble	Python (Not explicitly stated, but the paper mentions Python as the primary tool)	Python	Seven datasets from the NASA MDP repository (CM1, JM1, MC2, MW1, PC1, PC3, and PC4)	Predicted Positive Value (PPV), Predicted Negative Value (PNV), True Positive Rate (TPR), True Negative Rate, accuracy, Misclassification Rate (MR), False Positive Ratio (FPR), False Negative Ratio (FNR)	Split data into training and testing sets (70:30 ratio, class-based splitting)	Achieving better results than twenty top-tier defect prediction models.
9	LogReg, RF, Decision Stump, SVM, BN, MLP, J48, Lazy IBK	WEKA, Minitab	Not specified.	Five datasets from NASA PROMISE repository (CM1, JM1, KC1, KC2, PC1)	Accuracy	Cross-validation (10-fold and 30-fold)	LogReg achieved the highest accuracy of 93%

Among the 30 journal papers analyzed, while most primarily focus on class imbalance in datasets and on high computational complexity, the remaining studies explore a range of other challenges in SDP.

The study by Stradowski, S., & Madeyski, L. [9], examines the business applicability of Machine Learning-based Software Defect Prediction (ML SDP) in industry, particularly within Nokia's 5G software testing. Unlike many academic works, it highlights challenges in real-world adoption, such as high complexity, low understandability, and costly maintenance. The systematic literature review identifies methods, features, and frameworks but finds little focus on cost-effectiveness and practical implementation. The study emphasizes the need for structured user feedback, mixed dataset validation, and return on investment (ROI) assessments to bridge the gap between academic research and industry adoption.

Stradowski et al. [10] presents a systematic mapping of ML SDP from a business perspective, evaluating its adoption in commercial settings. The paper highlights the dominance of academic research over industry-based studies, with most models relying on open-source and NASA datasets rather than real-world commercial data. The study identifies emerging trends in ML SDP but emphasizes that high complexity, low understandability, and expensive maintenance hinder its adoption in business environments. By mapping 742 relevant studies, the authors provide insight into future research directions and stress the need for industry-driven validation of ML SDP models.

Aquil et al. [12] explore various ML techniques, including ensemble methods, clustering, and classification, to enhance SDP. The paper emphasizes the importance of early defect detection for improving software reliability and reducing maintenance costs. By evaluating multiple predictor models on 13 software defect datasets, the study identifies ensemble techniques, particularly stacking multiple classifiers, as the most effective approach for consistent high-accuracy predictions.

Geanderson Santos et al. [13] employed an XGBoost-based model for software defect prediction, analyzing millions of randomly generated models. Their study revealed that only 3.5% of the models achieved higher AUC values than the baseline, confirming that SDP is project-specific. The research also demonstrated that models with fewer, well-distributed features perform better and are easier to interpret. Using SHAP values, they identified key features influencing defect predictions, highlighting the need for explainable ML models in software development.

Batool and Khan [19] conducted a SLR analyzing data mining, ML, and DL techniques for SDP. The study examined 68 primary research papers, categorizing them based on datasets, methodologies, and performance measures. The review found that traditional data mining and ML techniques are widely used in defect prediction, while ensemble methods and DL approaches are less frequently employed. The study differentiates between each approach, which indirectly hints at computational complexity differences. However, it does not provide an in-depth analysis of computational costs associated with different techniques. The authors emphasized the need for more standardized datasets and further research on applying consistent techniques across different datasets to improve prediction performance.

Geanderson Esteves et al. [23] focuses on SDP using the XGBoost algorithm, with an emphasis on improving model interpretability. The authors propose a model sampling approach to identify the minimum set of relevant software features necessary for accurate defect prediction which may improve computational efficiency. However, it does not explicitly analyze computational complexity trade-offs. The study evaluates models on the Jureczko dataset and finds that project-specific factors influence defect prediction performance. Additionally, SHAP values are used to explain model decisions, ensuring that the predictions are interpretable and useful for developers. Future work suggests analyzing public GitHub repositories and developing tools to support real-world software projects.

Shikha Gautam et al. [29] focuses on SDP utilizng various ML algorithms (DT, NB, KNN, SVM, and RF). The experiment was conducted on the KC2 dataset from the NASA PROMISE repository, demonstrating that Naïve Bayes and SVM performed better in predicting software defects. It also highlights the challenges posed by noisy features and high-dimensional data in the dataset. Eventhough this study discusses working with high-dimensional data and testing different ML models, which indirectly relates to computational complexity, however it does not explicitly address algorithm efficiency or runtime concerns.

Last but not least, Nowrin Muhaimin Shailee et al. [30] and colleagues come up with a SDP model using ML techniques like NB, DT, RF, SVM, LogReg, ANN, and KNN with the NASA JM1 dataset. The results highlight RF as the most effective model with the highest accuracy (81%) and lowest RMSE. The research emphasizes the importance of early defect detection in software development and suggests future improvements, such as advanced feature engineering and incorporating domain-specific knowledge. The study compares different ML models, indirectly touching on computational efficiency but does not explicitly address algorithmic complexity or resource constraints.

Using RF and SVM for SDP offers several advantages, making them the preferred choice based on various literature reviews. RF is known for its top-tier accuracy and robustness, especially dealing with large datasets that have numerous features. By combining multiple decision trees, RF improves prediction accuracy and controls over-fitting, which is crucial for effective defect prediction. SVM, on the other hand, excels in high-dimensional spaces and is particularly effective when the quantity of dimensions surpasses the quantity of samples. It aims to identify the optimal hyperplane that effectively differentiates between the classes, ensuring accurate defect prediction. One of the advantages of RF is its ability to provide insights into feature importance, helping to identify which features contribute most to defect prediction. This is valuable for feature selection and dimensionality reduction. SVM is also effective in handling sparse data and plays a crucial role in scenarios where feature selection is necessary to enhance model performance. Addressing class imbalance is another significant advantage of these algorithms. RF can handle imbalanced datasets by adjusting class weights, making it

suitable for SDP where defects are often less frequent than non-defects. Similarly, SVM can be adapted to handle class imbalance by adjusting class weights, ensuring that the model does not prioritize the predominance class. Combining RF and SVM leverages the strengths of both algorithms. RF's ability to handle large datasets and provide feature importance complements SVM's effectiveness in high-dimensional spaces and strong classification capabilities. This combination offers a more comprehensive and accurate defect prediction model by addressing the limitations of each algorithm individually. Empirical evidence supports the superiority of combining RF and SVM for SDP. Studies have shown that this combination outperforms other algorithms in performance metrics including accuracy, AUC, and recall. By leveraging the strengths of both algorithms, the proposed SDP model achieves high accuracy, robustness, and reliability, effectively identifying defect-prone modules and enhancing software quality.

The literature review highlights several gaps in the existing research on SDP using ML techniques. While significant progress has been made in addressing class imbalance and computational complexity, there remains a need for more comprehensive studies that integrate these aspects into hybrid models. The proposed hybrid ML model aims to fill these gaps by combining effective feature selection techniques with robust RF and SVM methods, ultimately enhancing predictive accuracy and interpretability in SDP.

The literature review examined various ML techniques for SDP, focusing on two critical challenges: class imbalance and computational complexity. Several methods, including oversampling (SMOTE, COSTE) and ensemble learning (Bagging, Boosting), have been explored to mitigate these issues. While individual models such as RF and SVM have demonstrated effectiveness, their limitations highlight the need for hybrid approaches. The review underscores a research gap in integrating hybrid models with optimized feature selection and scalable architecture. This chapter sets the foundation for the proposed RF-SVM model, designed to improve predictive performance while addressing these challenges.

III. RESEARCH METHODOLOGY

In this chapter, the approach of solving problems of SDP was presented with the proposed solution methods and the progress achieved thus far. Research methodology serves as the foundation for systematically conducting investigations, guiding the processes throughout the project. The methodology implemented in this research, which is shown in Figure 3.1, includes key components: data collection, data analysis, data preprocessing, hybrid model development, the training and evaluation of the proposed model, validation, and verification.

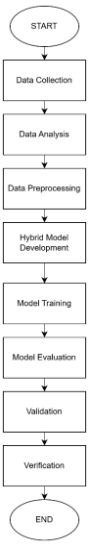


Fig. 1. Research Methodology

The inception of the methodology begins with the data collection phase. It involves gathering and preparing numerical data from open-source repositories. This includes gathering relevant data sets that contain instances of software defects and labeling them based on time domain limitations. Careful attention is given to ensuring that the data is comprehensive and representative of the various types of defects that may occur in software development. Data analysis involves creating charts and graphs to visually represent the collected data, alongside generating summary statistics to understand the data's distribution and characteristics. This step is crucial to find patterns, trends, and anomalies inside the data, serving as a baseline for the next preprocessing and model training. Next, the data preprocessing step requires cleaning and normalizing the data to ensure its quality and consistency. This step includes removing any irrelevant or redundant information, handling missing values, and adjusting the data to a consistent range. Feature selection techniques are then used to pinpoint the most pertinent features for the prediction model, enhancing its accuracy and efficiency. The development of a hybrid model involves combining the strengths of both the RF and SVM models to create an extra robust, strong, and accurate prediction model. By harnessing the complementary capabilities of each model, the hybrid approach aims to improve the entire predictive capability and address limitations of individual models. Model training entails training both the RF and SVM models using the preprocessed data. Each model is trained to learn from existing data and identify patterns that indicate the presence of software defects. This phase focuses on optimizing the models' parameters to maximize their predictive capabilities. Following next, the trained models are assessed by utilizing many performance metrics, in this case, accuracy, precision, recall, F1-score, and AUC-ROC. These measurements give a complete picture of how well each model predicts software faults. A comparative analysis of the RF and SVM models is performed to determine their respective strengths and weaknesses. The validation phase is intended to assess the generalization performance of the hybrid model on unseen data. This will be achieved through a hold-out validation strategy, typically by splitting the dataset into training and testing subsets (e.g., 80:20). The model will be trained on one portion of the data and validated on the other, ensuring that its

performance is not biased by the training process. The goal of this step is to confirm that the model can reliably detect defects in new, real-world software modules. The final phase which is verification involves checking data handling, feature processing, and model integration to confirm that the system behaves as intended. It will help maintain consistency, correctness, and reproducibility throughout the development process. This structured approach ensures a thorough and effective methodology for predicting software defects, leveraging the strengths of ML techniques to enhance software quality and reduce development costs.

Implementation Progress

This project uses Python as the primary programming language, with Visual Studio for model development, analysis, and testing.

A. Data Collection

The foundation of this research lies in the acquisition of high-quality, representative data relevant to software defect prediction. For this purpose, the JM1 dataset was selected from the publicly available NASA PROMISE dataset repository, a trusted benchmark in empirical software engineering studies. The dataset source is listed in the References section.

i. Dataset Description

The JM1 data contains program measures obtained from a real NASA ground system project written in the C programming language. It contains 10,885 software components with a binary defect label indicating whether the module is defective (1) or not (0). The binary character renders the data suitable for supervised learning tasks, especially for training and cross-validation of machine learning classifiers to detect software defects. The dataset contains 22 features, derived from widely cited McCabe and Halstead complexity metrics, line-of-code metrics and branch metrics. The metrics have been widely used in software quality analysis due to their theoretical background and empirical relevance. The features in the dataset are divided into four general categories:

1. McCabe Metrics

These metrics evaluate the control flow structure and structural complexity of software modules:

TABLE III. MCCABE METRICS

Attributes	Description
Cyclomatic Complexity (v(G))	Measures the number of linearly independent paths through a module. High values often correlate with increased defect likelihood.
Essential Complexity (ev(G))	Indicates the extent of unstructured logic, reflecting code maintainability.
Design Complexity (iv(G))	Measures complexity related to inter-module interactions.
Lines of Code (loc)	Total lines of code, commonly used as a baseline metric for size and defect risk.

2. Halstead Metrics

These metrics quantify computational complexity using operators and operands:

TABLE IV. HALSTEAD METRICS

Attributes		Description
Base Measures (Direct counts from code)	uniq_Op	Number of unique operators (e.g., +, -, if, return). Indicates the variety of operations used.
	uniq_Opnd	Number of unique operands (e.g., variables, constants). Reflects code diversity and input usage.
	total_Op	Total number of operator occurrences. High values may imply complex logic or branching.
	total_Opnd	Total number of operand occurrences. Larger values suggest more variables/data used.
Derived Measures (Calculated using Halstead's formula, these reflect code complexity, maintainability, and developer effort.)	n	Total tokens (n = total_Op + total_Opnd), indicates program length at the lexical level.
	v	Volume: v = n * log2(uniq_Op + uniq_Opnd), represents the size of the implementation.
	l	Program length (estimated) can be used to approximate how verbose or

		dense the code is.
	d	Difficulty: d = (uniq_Op / 2) * (total_Opnd / uniq_Opnd), quantifies how hard the code is to understand or change.
	i	Intelligence content indicates the understandability or the design quality of the module.
	e	Effort: e = v * d, an estimate of the effort needed to write or understand the code.
	t	Time estimation: t = e / 18 (seconds), the estimated time in seconds for implementation or comprehension.
	b	Estimated bugs: b = e^(2/3) / 3000, predicts potential defect count based on effort.
Line-Based Measures (Derived from physical characteristics of the source code.)	lOCode	Lines of code (non-blank, non-comment). directly reflects implementation length.
	lOComment	Lines of comments, important for maintainability and code documentation quality.
	lOBlank	Blank lines, contributes to code readability and structure
	lOCodeAndComment	Sum of code and comment lines, can indicate codebase documentation quality and size.

3. Control Flow Metric

TABLE V. CONTROL FLOW METRIC

Attribute	Description
branchCount	Representing the number of branches in the module's control flow graph.

4. Target Variable

TABLE VI. TARGET VARIABLE

Attribute	Description
defects	A binary label indicating whether the module contains one or more known defects (true) or is defect-free (false).

A significant characteristic of the JM1 dataset is its class imbalance:

- Defective modules (true): 80.65%
- Non-defective modules (false): 19.35%

Such imbalance is common in defect prediction problems and presents a challenge for most classification models. To address this, later preprocessing stages incorporate techniques like resampling or class-weight tuning to improve model fairness and predictive performance.

These metrics collectively capture syntactic complexity, logical structure, and maintainability characteristics of software modules, making the dataset highly suitable for software defect prediction using machine learning. These software metrics together form a comprehensive profile of each module and serve as the independent variables used for model training. Table 3.5 shows the order of attributes in the dataset.

TABLE VII. ATTRIBUTES IN JM1 DATASET

No	Attributes	Information
1	Loc	numeric % McCabe's line count of code
2	v(g)	numeric % McCabe "cyclomatic complexity"
3	ev(g)	numeric % McCabe "essential complexity"

4	iv(g)	numeric % McCabe "design complexity"
5	n	numeric % Halstead total operators + operands
6	v	numeric % Halstead "volume"
7	l	numeric % Halstead "program length"
8	d	numeric % Halstead "difficulty"
9	i	numeric % Halstead "intelligence"
10	e	numeric % Halstead "effort"
11	b	numeric % Halstead
12	t	numeric % Halstead's time estimator
13	lOCode	numeric % Halstead's line count
14	lOComment	numeric % Halstead's count of lines of comments
15	lOBlank	numeric % Halstead's count of blank lines
16	lOCodeAndComment	numeric
17	uniq_Op	numeric % unique operators
18	uniq_Opnd	numeric % unique operands
19	total_Op	numeric % total operators
20	total_Opnd	numeric % total operands
21	branchCount	numeric % of the flow graph
22	defects	{false,true} % module has/has not one or more reported defects

ii. Evaluation Metrics for Defect Prediction

To measure the effectiveness of the proposed defect prediction models, several well-established performance evaluation metrics are used. These metrics are based on a confusion matrix that compares predicted defect labels to the actual defect status of each software module:

TABLE VIII. EVALUATION METRICS FOR SDP

		module actually has defects	
		no	yes
classifier predicts no defects	no	a	b
classifier predicts some defects	yes	c	d

iii. Data Format and Loading Procedure

The dataset for this research originates from the NASA JM1 dataset. Comprising a substantial collection of software defect data, it includes instances that are essential for our study. The dataset encompasses numerous samples categorized into different defect types, making it well-suited for training and testing our RF-SVM hybrid model. The dataset's structure includes columns that represent various software metrics and labels indicating the presence or absence of defects. Figure 3.2 shows the pseudocode to read the dataset, and Figure 3.3 shows some samples from the dataset.

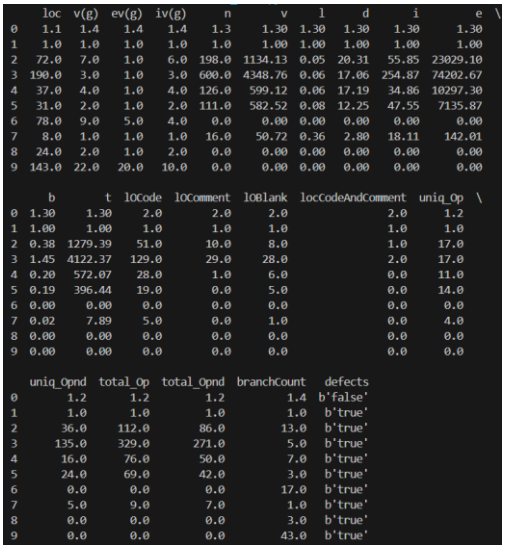
START

1. Import required libraries:
   a. pandas for data handling
   b. scipy.io.arff for reading ARFF files
2. Load the dataset:
   a. Use the arff.loadarff() function to read the "jm1.arff" file
   b. Convert the result into a pandas DataFrame
3. Configure display settings:
   a. Set pandas to display all columns
4. Inspect the dataset:
   a. Use DataFrame.head() to preview the first few rows

END

Fig. 2. Pseudocode for Reading ARFF Dataset

The program ensures that the dataset is correctly loaded and formatted, with all attributes accessible for inspection and preprocessing. Initial checks confirmed that the dataset is complete, contains no missing values, and consists entirely of numerical attributes, making it highly compatible with standard machine learning models.



```
    loc  v(g)  ev(g)  iv(g)     n       v    l     d     i       e \
0   1.1   1.4    1.4    1.4   1.3    1.30 1.30  1.30  1.30    1.30
1   1.0   1.0    1.0    1.0   1.0    1.00 1.00  1.00  1.00    1.00
2  72.0   7.0    1.0    6.0 198.0 1134.13 0.05 20.31 55.85 23029.10
3 190.0   3.0    1.0    3.0 600.0 4348.76 0.06 17.06 254.87 74202.67
4  37.0   4.0    1.0    4.0 126.0  599.12 0.06 17.19 34.86 10297.30
5  31.0   2.0    1.0    2.0 111.0  582.52 0.08 12.25 47.55  7135.87
6  78.0   9.0    5.0    4.0   0.0    0.00 0.00  0.00  0.00     0.00
7   8.0   1.0    1.0    1.0  16.0   50.72 0.36  2.80 18.11   142.01
8  24.0   2.0    1.0    2.0   0.0    0.00 0.00  0.00  0.00     0.00
9 143.0  22.0   20.0   10.0   0.0    0.00 0.00  0.00  0.00     0.00

      b       t  lOCode  lOComment  lOBlank  locCodeAndComment  uniq_Op  \
0  1.30    1.30     2.0        2.0      2.0                2.0      1.2
1  1.00    1.00     1.0        1.0      1.0                1.0      1.0
2  0.38 1279.39    51.0       10.0      8.0                1.0     17.0
3  1.45 4122.37   129.0       29.0     28.0                2.0     17.0
4  0.20  572.07    28.0        1.0      6.0                0.0     11.0
5  0.19  396.44    19.0        0.0      5.0                0.0     14.0
6  0.00    0.00     0.0        0.0      0.0                0.0      0.0
7  0.02    7.89     5.0        0.0      1.0                0.0      4.0
8  0.00    0.00     0.0        0.0      0.0                0.0      0.0
9  0.00    0.00     0.0        0.0      0.0                0.0      0.0

   uniq_Opnd  total_Op  total_Opnd  branchCount   defects
0        1.2       1.2         1.2          1.4  b'false'
1        1.0       1.0         1.0          1.0   b'true'
2       36.0     112.0        86.0         13.0   b'true'
3      135.0     329.0       271.0          5.0   b'true'
4       16.0      76.0        50.0          7.0   b'true'
5       24.0      69.0        42.0          3.0   b'true'
6        0.0       0.0         0.0         17.0   b'true'
7        5.0       9.0         7.0          1.0   b'true'
8        0.0       0.0         0.0          3.0   b'true'
9        0.0       0.0         0.0         43.0   b'true'
```

Fig. 3. Output of Reading ARFF File

This figure shows the command-line output after executing the Python script jm1.py, which is the file name for the JM1 dataset. The output displays the first 10 rows of the DataFrame, which contains 22 columns. The purpose of this command-line output is to verify that the dataset has been successfully loaded into the DataFrame. By displaying the first few rows of the dataset, we can ensure that the data is correctly formatted and all set for further examination and preprocessing. This step is crucial for validating the data loading process and ensuring that the subsequent steps in the research methodology are based on accurate and reliable data.

B. Data Analysis

To gain an initial understanding of the NASA JM1 dataset, an exploratory data analysis (EDA) is performed to summarize the main characteristics of the data.

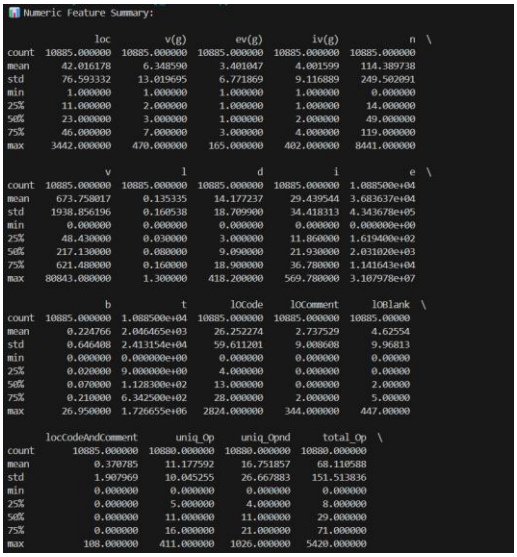
i. Summary Statistic

The summary statistics provide a quick summary of measures of the central tendency, dispersion, and shape of the distribution of the features. Figure 3.4 shows the pseudocode, while Figure 3.5 and 3.6 below showcases the summary statistics for various metrics related to LOC and other software attributes in the dataset. These metrics include LOC_BLANK, BRANCH_COUNT, LOC_CODE_AND_COMMENT, NUM_UNIQUE_OPERATORS, LOC_TOTAL, and several others. The summary statistics cover several metrics, like the count, mean, standard deviation, minimum, 25th percentile (Q1), median (50th percentile), 75th percentile (Q3), and maximum values.

```
START
1. Import the required libraries:
   a. pandas for data handling
   b. scipy.io.arff for loading ARFF
      files
2. Load the dataset from the specified file
   path using arff loader.
3. Convert the data into a DataFrame
   structure for easier manipulation.
4. For each column with byte-type data:
        a. Decode it from byte format
           (e.g., b'true') into standard
           UTF-8 strings.
5. Rename the 'class_attribute' column to
   'class' for consistency (if it exists).
6. If a 'class' column exists:
   a. Convert its values to lowercase
      strings.
   b. Map the string labels: 'false' → 0
      and 'true' → 1.
   c. Store the result in a new column
      called 'defects'.
7. Display all available columns in the
   output (remove column visibility
   limits).
8. Print descriptive statistics (mean,
   std, min, max, etc.) for all numeric
   columns.
9. Print summary statistics (count,
   unique, top, freq) for all object or
   boolean columns.
END
```

Fig. 4. Pseudocode for Summary Statistic

The code begins by importing essential libraries and loading an ARFF-formatted dataset, a structure commonly associated with software engineering data from NASA's PROMISE repository. Once loaded, the data is stored in a pandas DataFrame to simplify subsequent operations such as filtering, transformation, and statistical analysis. Because ARFF files often encode strings as byte literals (e.g., b'true'), decoding is necessary to ensure compatibility with standard string operations in Python. The class label is then cleaned and standardized, and renamed to class if needed, then mapped to binary values (0 for non-defective and 1 for defective), under a new column named defects, preparing it for use in supervised machine learning. The script also ensures full column visibility during output display. Next, it generates two sets of descriptive statistics: one for numeric features, covering metrics such as mean, standard deviation, and percentiles, and one for categorical features, including counts, unique value counts, the most frequent category, and its frequency.



Numeric Feature Summary:

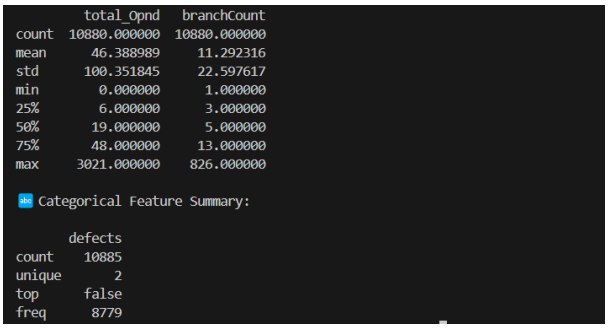
	loc	v(g)	ev(g)	iv(g)	n
count	10885.000000	10885.000000	10885.000000	10885.000000	10885.000000
mean	42.016178	6.348590	3.401047	4.001599	114.389738
std	76.593332	13.019695	6.771869	9.116889	249.502091
min	1.000000	1.000000	1.000000	1.000000	0.000000
25%	11.000000	2.000000	1.000000	1.000000	14.000000
50%	23.000000	3.000000	1.000000	2.000000	49.000000
75%	46.000000	7.000000	3.000000	4.000000	119.000000
max	3442.000000	470.000000	165.000000	402.000000	8441.000000

	v	l	d	i	e
count	10885.000000	10885.000000	10885.000000	10885.000000	1.088500e+04
mean	673.758017	0.135335	14.177237	29.439544	3.683637e+04
std	1938.856196	0.160538	18.709900	34.418313	4.343678e+05
min	0.000000	0.000000	0.000000	0.000000	0.000000e+00
25%	48.430000	0.030000	3.000000	11.860000	1.619400e+02
50%	217.130000	0.080000	9.090000	21.930000	2.031020e+03
75%	621.480000	0.160000	18.980000	36.780000	1.141643e+04
max	80843.080000	1.300000	418.200000	569.780000	3.107978e+07

	b	t	lOCode	lOComment	lOBlank
count	10885.000000	1.088500e+04	10885.000000	10885.000000	10885.00000
mean	0.224766	2.046465e+03	26.252274	2.737529	4.62554
std	0.646408	2.413154e+04	59.611201	9.008608	9.96813
min	0.000000	0.000000e+00	0.000000	0.000000	0.00000
25%	0.020000	9.000000e+00	4.000000	0.000000	0.00000
50%	0.070000	1.128300e+02	13.000000	0.000000	2.00000
75%	0.210000	6.342500e+02	28.000000	2.000000	5.00000
max	26.950000	1.726655e+06	2824.000000	344.000000	447.00000

	locCodeAndComment	uniq_Op	uniq_Opnd	total_Op
count	10885.000000	10880.000000	10880.000000	10880.000000
mean	0.370785	11.177592	16.751857	68.110588
std	1.907969	10.045255	26.667883	151.513836
min	0.000000	0.000000	0.000000	0.000000
25%	0.000000	5.000000	4.000000	8.000000
50%	0.000000	11.000000	11.000000	29.000000
75%	0.000000	16.000000	21.000000	71.000000
max	108.000000	411.000000	1026.000000	5420.000000

Fig. 5. Output of Summary Statistic (i)



	total_Opnd	branchCount
count	10880.000000	10880.000000
mean	46.388989	11.292316
std	100.351845	22.597617
min	0.000000	1.000000
25%	6.000000	3.000000
50%	19.000000	5.000000
75%	48.000000	13.000000
max	3021.000000	826.000000

Categorical Feature Summary:

	defects
count	10885
unique	2
top	false
freq	8779

Fig. 6. Output of Summary Statistic (ii)

ii. Correlation Matrix

A correlation matrix is a table that shows the pairwise correlation coefficients between variables in the dataset. The correlation coefficient is between -1 to 1, where a correlation value of 1 indicates a perfect positive relationship, 0 denotes no linear relationship, and -1 indicates a perfect negative

relationship between variables. The heatmap visualizes these correlation coefficients, where the color intensity signifies the strength of the correlation. Strong positive correlations are shown by bright red, strong negative correlations by bright blue, and weak correlations by lighter colors.

By examining the correlation matrix, it is easy to identify which metrics have strong linear relationships with each other. This information is useful for feature selection, multicollinearity analysis, and understanding the dataset's underlying structure.

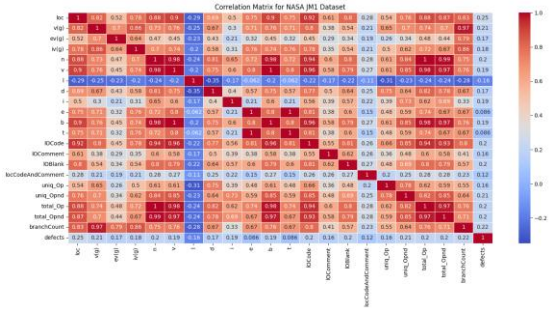


Fig. 7. Correlation Matrix for JM1 Dataset

iii. Feature Selection

1. Correlation with Target

Feature selection is very important in machine learning, especially in high-dimensional datasets such as the JM1 software defect dataset. The goal is to identify the most relevant input features (metrics) that contribute to predicting the target label, in this case, whether a software module is defective (1) or not (0). This not only reduces model complexity and training time but can also improve model performance by removing irrelevant or redundant attributes.

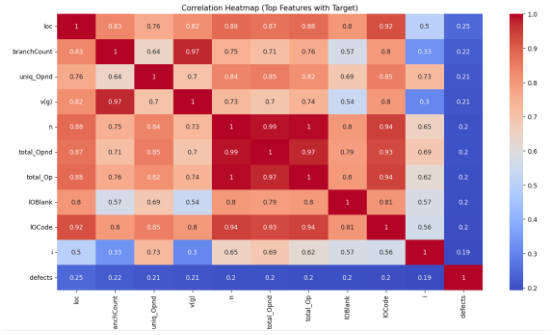


Fig. 8. Correlation Heatmap (Top Features with Target)

```
Correlation of features with 'defects':
loc                     0.245437
branchCount             0.222754
uniq_Opnd               0.211952
v(g)                    0.208670
n                       0.204087
total_Opnd              0.203825
total_Op                0.200987
lOBlank                 0.199489
lOCode                  0.195857
i                       0.192643
b                       0.189318
v                       0.189113
iv(g)                   0.181981
ev(g)                   0.172935
d                       0.169442
l                       0.164721
lOComment               0.161131
uniq_Op                 0.158834
locCodeAndComment       0.118054
t                       0.086092
e                       0.086092
Name: defects, dtype: float64
```

Fig. 9. Correlation of features with defects

From the analysis, it's found that loc (Lines of Code) and branchCount are the most strongly correlated with the presence of defects. Other significant metrics include Halstead base measures such as uniq_Opnd, total_Opnd, and total_Op, as well as McCabe complexity metrics like v(g) (Cyclomatic Complexity) and iv(g) (Design Complexity) are moderate. Derived Halstead metrics such as b (bugs), i (intelligence content), d (difficulty), and e (effort) also show moderate correlation. Although correlation alone doesn't imply causation, it provides a good initial insight into which attributes may influence software defectiveness. These findings were used to guide further feature selection in model development.

2. Feature Importance

In addition to correlation analysis, feature importance was examined to assess which software metrics are most influential in determining defectiveness. This process helps identify variables that may significantly contribute to the predictive performance of the machine learning models. A Random Forest Classifier was trained on the dataset to compute feature importances based on how frequently and effectively each feature was used in decision splits. Random Forests are especially suitable for this task due to their ensemble structure and built-in feature ranking capabilities.

The top features identified include:

1. **loc (Lines of Code):** Higher code length generally indicates complexity, which may increase defect likelihood.
2. **branchCount:** More branching may introduce logical complexity.
3. **uniq_Opnd** and **uniq_Op:** Halstead base measures indicating the richness of operands and operators in the code.
4. **v(g) (Cyclomatic Complexity):** Measures the number of linearly independent paths through the code.

These findings support the intuition that complexity-related metrics (like size, branching, and token diversity) correlate strongly with the presence of software defects.

The resulting feature importance values were later leveraged during hybrid model development to reduce dimensionality and optimize the SVM component by focusing on the top-ranked predictors.

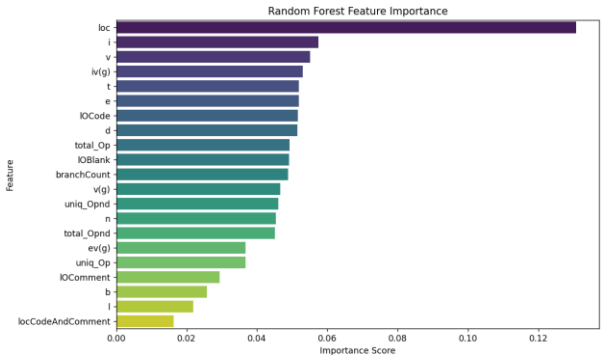


Fig. 10. Random Forest Feature Importance

iv. Data Visualization

Data visualization is a crucial step in the analysis pipeline, as it provides intuitive insights into the structure, distribution, and relationships within the dataset. By visually exploring the features and their interactions, potential anomalies, trends, and class imbalances can be easily identified. In this project, key visualizations include pair plots to understand feature-to-feature and feature-to-class relationships, as well as a class distribution plot to evaluate the balance between defective and non-defective instances. These visual analyses guide preprocessing decisions and model selection by revealing patterns that may affect learning performance.

1. Class Distribution

The class distribution analysis for the JM1 dataset reveals a significant class imbalance. Out of a total of 10,885 samples, 8,779 instances (≈80.7%) are labeled as non-defective (0), while only 2,106 instances (≈19.3%) are labeled as defective (1). This results in an imbalance ratio of approximately 4.2:1, indicating that non-defective samples vastly outnumber the defective ones. Additionally, statistical measures of the defects column confirm this skew, with a mean of 0.193, implying that fewer than 1 in 5 samples are defective. Such imbalance can negatively impact model performance, especially in detecting minority class instances, and therefore justifies the use of resampling techniques like SMOTE or applying class weights during model training.

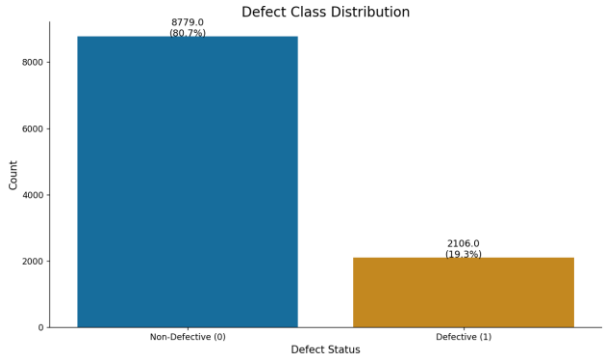


Fig. 11. JM1 Class Distribution

```
Class Distribution Summary:
Non-Defective: 8779 samples (80.7%)
Defective: 2106 samples (19.3%)
Imbalance Ratio: 4.2:1

Additional Statistics:
count    10885.000000
mean         0.193477
std          0.395042
min          0.000000
25%          0.000000
50%          0.000000
75%          0.000000
max          1.000000
Name: defects, dtype: float64
```

Fig. 12. Class Distribution Summary

C. *Data Preprocessing*

Although the JM1 dataset used in this project was sourced from a public GitHub repository in a structured .arff format, additional preprocessing was necessary to prepare it for effective machine learning modeling. The dataset, originally developed by NASA under the Metrics Data Program (MDP), includes various software metrics derived from McCabe and Halstead complexity measures, along with a binary class label indicating the presence or absence of defects.

The first preprocessing task involved decoding the class label field, which was stored as a byte string (e.g., b'true' and b'false'). These were converted to regular string values and subsequently mapped to numeric representations (1 for defective modules and 0 for non-defective modules) to ensure compatibility with scikit-learn's classification algorithms.

```
1. Load the dataset from the ARFF file.
2. Convert the dataset into a pandas
   DataFrame.
3. Identify the 'defects' column, which
   contains byte strings like b'true' and
   b'false'.
4. Decode the byte strings into standard
   Python strings.
5. Map the string values:
   a. 'true' → 1 (defective)
   b. 'false' → 0 (non-defective)
6. Replace the original 'defects' column
   with the numeric version.
```

Fig. 13. Pseudocode for Decoding class label

Following this, the dataset was inspected for missing values. Based on both the documentation and programmatic checks, no missing or null values were found, and thus no imputation or removal operations were needed. The dataset's attributes were also confirmed to be entirely numerical, so no categorical encoding was required.

```
CHECK the dataset for any null or
missing values

IF missing values exist:

    HANDLE them using imputation or
removal

ELSE:

    CONTINUE
```

Fig. 14. Pseudocode for handling missing value

Next, the features and class labels were separated into input matrix X and output vector y, respectively. This separation is crucial for supervised learning tasks, enabling the model to learn patterns from the software metrics (X) that correspond to defect labels (y).

```
ASSIGN all columns except 'defects' to
variable X
ASSIGN the 'defects' column to variable
y
```

Fig. 15. Pseudocode for splitting features and labels

Although basic data cleaning was minimal due to the preprocessed nature of the source file, additional scaling of features was later applied during model training using a RobustScaler. This step ensured that the models, especially SVM, which is sensitive to input scales could operate more effectively by reducing the influence of outliers and normalizing feature ranges.

```
INITIALIZE a RobustScaler
FIT the scaler to X and TRANSFORM it
STORE the scaled values in a new
variable
```

Fig. 16. Pseudocode for Feature Scaling with RobustScaler

In summary, while the dataset did not require extensive raw cleaning, several essential preprocessing steps were performed to ensure the data was clean, consistent, and compatible with the downstream modeling pipeline.

D. Hybrid Model Development and Model Training

To evaluate and improve software defect prediction performance, four model configurations were developed and compared. The first two served as baselines using individual classifiers: RF and SVM. The remaining two configurations implemented hybrid models that combined their strengths. The first hybrid model RF-SVM stacked RF and SVM predictions to form a simple ensemble, while the second RF-SVM+ extended this further with additional enhancements such as probability calibration and meta-learning.

This phase also includes the training process, which is inherently embedded within the hybrid development workflow. Each component (RF, SVM, and the meta-classifier) is trained sequentially during development using preprocessed and balanced data. SMOTE is first applied to address class imbalance, followed by feature selection via RF. The top-ranked features are then used to train a fast linear SVM, which is enhanced through Platt scaling to generate calibrated probabilities. Finally, the hybrid architecture integrates RF and SVM outputs (including their interaction) into a Gradient Boosting meta-model, which is trained to make the final predictions. Thus, model training and development are executed in a single unified process.

a) Model 1 - RF only

RF is an ensemble learning method that constructs multiple decision trees and aggregates their predictions to improve classification accuracy and reduce overfitting. It works well with imbalanced data and provides feature importance insights [34].

This model implementation utilizes the RF algorithm as a standalone classifier to predict software module defects using a preprocessed software metrics dataset. The objective of this model is to serve as a baseline to evaluate the predictive power of RF in comparison to more complex hybrid models.

The process begins with data loading and preprocessing. The input data, stored in .arff format, is read into a Pandas DataFrame. The target column, originally named defects, is renamed to label for clarity. Since .arff files can encode categorical values as byte strings, all such columns are decoded into UTF-8 strings. The target values are then mapped from string labels ('false' and 'true') to binary integers 0 and 1, respectively. Any instances containing missing values are removed to ensure a clean dataset.

Once cleaned, the data is divided into features (X) and labels (y). These are subsequently split into training and test subsets using an 80:20 ratio. This ensures that the model's performance is evaluated on unseen data, helping to gauge generalization capability.

To prepare the data for model training, feature scaling is applied using RobustScaler, which is particularly resilient to the presence of outliers. This scaler transforms the feature values by removing the median and scaling based on the interquartile range, ensuring that extreme values do not unduly influence the learning process.

The core of the model is a Random Forest classifier, instantiated with 50 decision trees (n_estimators=50) and a fixed random seed to ensure reproducibility. The model is trained on the scaled training set to learn patterns distinguishing defective from non-defective modules. Once trained, the model is evaluated on the test set by predicting class probabilities. These probabilities are converted to binary class predictions using a threshold of 0.5, where probabilities greater than or equal to 0.5 are classified as defective (1), and those below as non-defective (0).

The performance of the model is assessed using standard evaluation metrics: precision, recall, and F1-score for both classes, alongside the ROC-AUC score which captures the overall separability between the classes. These metrics provide detailed insights into the model's ability to detect both defective and non-defective software modules accurately.

Beyond test set evaluation, the model is also applied to the entire dataset to generate predictions across all modules. This is done by scaling the full feature set using the previously fitted scaler, passing it through the trained Random Forest model to obtain defect probability estimates, and converting those probabilities into final binary predictions. The system then reports a summary of the predicted versus actual counts

of defective and non-defective modules in the dataset. This step provides a practical overview of how the model would behave in a real-world setting, where predictions need to be made for the entire software project.

In summary, the RF model offers a solid and interpretable baseline by leveraging its ensemble nature to reduce variance and overfitting while maintaining good generalization. Its strength lies in handling high-dimensional feature spaces, managing noisy data, and capturing non-linear interactions. The results from this model serve as a point of comparison against more advanced hybrid techniques, such as the RF-SVM stack, to assess improvements in defect prediction accuracy and robustness.

```
START

1. LOAD DATA
   - Read .arff file into DataFrame
   - Rename 'defects' → 'label'
   - Convert byte strings and map 'false'
→0, 'true'→1
   - Drop rows with missing values

2. SPLIT into features (X) and labels (y)

3. SPLIT into training and test sets
(80:20)

4. SCALE FEATURES
   - Use RobustScaler to reduce impact of
outliers

5. TRAIN Random Forest on training data

6. EVALUATE on test set
   - Predict probabilities → class labels
(threshold = 0.5)
   - Print precision, recall, F1-score,
ROC-AUC

7. PREDICT FULL DATASET
   - Apply scaler to full dataset
   - Predict defect probabilities
   - Convert to final predictions
(threshold = 0.5)
   - Count total predicted defects in
entire dataset

END
```

Fig. 17. Pseudocode for RF model

b) Model 2 – SVM Only

SVM is a powerful classifier that constructs a hyperplane to separate classes with maximum margin. It is effective in high-dimensional spaces but sensitive to input scale and class imbalance. It excels when dealing with datasets with limited data, drawing a decision boundary between classes to achieve maximum margins [22].

This model applies the SVM algorithm as a standalone classifier for software defect prediction using the JM1 software metrics dataset. The purpose of this model is to establish a comparative benchmark based on SVM's strengths in handling high-dimensional data and finding optimal decision boundaries.

The pipeline begins by loading and preprocessing the dataset stored in .arff format. The file is parsed into a Pandas DataFrame, and the column defects, which serves as the target variable, is renamed to label. Since .arff files often contain byte-encoded strings, all object-type columns are decoded into readable UTF-8 strings. The binary class labels 'false' and 'true' are then mapped to numerical values 0 (non-defective) and 1 (defective), respectively. To ensure data consistency, rows containing missing values are removed.

Following this, the data is split into feature variables (X) and target labels (y). An 80:20 train-test split is performed to separate the dataset into training and evaluation sets, ensuring that the model's performance can be fairly tested on previously unseen data.

The next critical step involves scaling the features using RobustScaler, a preprocessing technique that scales features according to their interquartile range. This method is effective for software engineering data, which often includes extreme values or outliers. By scaling the data, the model avoids being disproportionately influenced by anomalous feature values.

The SVM classifier is then instantiated using the Radial Basis Function (RBF) kernel, which allows the model to learn non-linear decision boundaries. Additionally, the probability=True parameter enables the SVM to output class probabilities, which are essential for soft classification decisions and for computing evaluation metrics like ROC-AUC. The model is trained on the scaled training set.

After training, the model is evaluated on the test set. It generates probability estimates for the defective class, which are then thresholded at 0.5 to obtain final binary predictions. The classification performance is summarized using standard metrics: precision, recall, and F1-score for both the non-defective and defective classes. Additionally, the ROC-AUC score is calculated to assess the model's overall ability to distinguish between the two classes across different threshold settings.

In the final step, the trained SVM model is applied to the entire dataset to generate defect predictions across all software modules. The full dataset is first transformed using the previously fitted scaler, and the model outputs defect probabilities for each instance. These are again thresholded at 0.5 to produce binary predictions. A summary report is generated comparing the total number of predicted defective and non-defective modules against the actual class distribution in the dataset.

In conclusion, the standalone SVM model leverages its strengths in learning complex, non-linear decision surfaces and handling high-dimensional data. It provides a robust baseline model for software defect prediction, particularly effective in datasets where class boundaries are not linearly separable. The performance metrics obtained from this model serve as an important point of reference for evaluating the improvements offered by hybrid or ensemble-based approaches such as RF-SVM or RF-SVM+.

```
START

1. LOAD DATA
```

```
   - Read .arff file into DataFrame
   - Rename 'defects' → 'label'
   - Convert byte strings and map 'false'
→0, 'true'→1
   - Drop rows with missing values

2. SPLIT into features (X) and labels (y)

3. SPLIT into training and test sets
(80:20)

4. SCALE FEATURES
   - Use RobustScaler to reduce impact of
outliers

5. TRAIN SVM (RBF kernel) on training data

6. EVALUATE on test set
   - Predict probabilities → class labels
(threshold = 0.5)
   - Print precision, recall, F1-score,
ROC-AUC

7. PREDICT FULL DATASET
   - Apply scaler to full dataset
   - Predict defect probabilities
   - Convert to final predictions
(threshold = 0.5)
   - Count total predicted defects in
entire dataset

END
```

Fig. 18. Pseudocode for SVM Model

c) Model 3 – RF-SVM Hybrid Model

This hybrid model implements a stacking ensemble strategy that integrates the strengths of RF and SVM, with a LogReg model acting as the meta-classifier. The approach begins by loading and preprocessing a preprocessed ARFF file containing software metrics and defect labels. The label column, originally named defects, is renamed to label, and string values are decoded and converted into binary format, which is mapping 'false' to 0 and 'true' to 1. Any rows with missing values are removed to ensure clean input data.

The dataset is then split into features and labels, followed by an 80:20 train-test split to enable unbiased evaluation. To ensure that the learning process is not skewed by extreme values in the data, the features are scaled using RobustScaler, which is particularly effective in reducing the influence of outliers.

Next, two base-level models are trained: a RF classifier with 50 trees, and an SVM model using an RBF kernel with probability outputs enabled. These models are both trained on the scaled training data. After training, both base models are used to generate class probability estimates for the test set, specifically focusing on the probability of the positive class (defective modules). These probabilities are then stacked side by side to form a new feature set, where each sample now contains two values: one from the RF and one from the SVM.

This new stacked feature set becomes the input for the meta-model, which in this implementation is a LogReg classifier. The LogReg model is trained using these stacked predictions alongside the actual test set labels. While it's more common to use out-of-fold predictions or a validation set for this step, here the test predictions are directly used to fit the meta-model. This decision simplifies the implementation but should be interpreted cautiously in a real-world deployment.

Once the meta-model is trained, it predicts probabilities on the stacked test set features. These probabilities are thresholded at 0.5 to generate final class predictions. The performance of this hybrid system is then evaluated using class-wise precision, recall, and F1-scores, as well as the overall ROC-AUC score. These metrics provide insight into the model's ability to correctly identify both defective and non-defective software modules.

Finally, the trained model is applied to the entire dataset. The full feature set is scaled and passed through the base models to obtain probability predictions, which are again stacked and processed by the meta-model to produce final defect predictions. The system outputs a summary comparing the total predicted defect and non-defect counts against the actual label distribution, offering a practical overview of the model's effectiveness when applied at scale.

Overall, this stacking approach enhances defect prediction performance by combining the complementary strengths of RF and SVM. RF contributes robustness to noise and non-linear patterns, SVM provides strong boundary-setting in high-dimensional space, and LogReg balances the predictions to improve overall decision-making. This layered learning structure allows the hybrid model to make more nuanced and accurate classifications than any individual model alone.

```
START
1. LOAD DATA
   - Read .arff file into DataFrame
   - Rename 'defects' → 'label'
   - Convert byte strings and map
'false'→0, 'true'→1
   - Drop rows with missing values
2. SPLIT into features (X) and labels (y)
3. SPLIT into training and test sets
(80:20)
4. SCALE FEATURES
   - Use RobustScaler to reduce impact of
outliers
5. TRAIN Base Models (Random Forest and
SVM)
6. STACK Predictions from RF and SVM →
input for meta-model
7. TRAIN Meta-Model (Logistic Regression)
8. EVALUATE on test set
   - Predict probabilities → class labels
(threshold = 0.5)
   - Print precision, recall, F1-score,
ROC-AUC
9. PREDICT FULL DATASET
   - Stack RF and SVM probabilities
   - Predict defect probabilities using
meta-model
   - Count total predicted defects in
entire dataset
```

```
END
```

Fig. 19. Pseudocode for Hybrid RF-SVM Model

d) Model 4 – RF-SVM+ (Enhanced Hybrid Model)

This implementation extends the hybrid RF–SVM model by incorporating additional enhancements to improve classification accuracy and robustness in the task of software defect prediction. The enhancements include data balancing via SMOTE, feature selection based on RF importance, Platt scaling for SVM calibration, and a meta-model (Gradient Boosting) trained on engineered features, including the absolute difference between RF and SVM predictions. This results in a more refined architecture referred to as RF+SVM+.

The workflow begins with preprocessing the JM1 dataset. The .arff file is loaded into a Pandas DataFrame, and the label column defects is renamed to label. Binary labels are converted to integers: 'true' becomes 1 (defective) and 'false' becomes 0 (non-defective). Missing values are removed to ensure data integrity.

To address class imbalance issues in the dataset, SMOTE (Synthetic Minority Oversampling Technique) is applied. This method synthesizes new samples for the minority class, effectively balancing the dataset. To reduce computational overhead, the resampled dataset is then randomly downsampled to 50% of its original size while maintaining the class distribution.

Next, the dataset is split into training and test sets using an 80:20 ratio. Feature scaling is applied using RobustScaler, which is known to perform well in the presence of outliers, ensuring more stable model training.

A RF classifier is trained on the full feature set, with class weights adjusted to emphasize defective instances. After training, feature importance scores are extracted, and the top 8 most important features are selected for downstream SVM modeling. These selected features are then scaled separately.

A Linear Support Vector Classifier (LinearSVC) is trained on the scaled, reduced feature set. Since LinearSVC does not natively produce probabilities, Platt Scaling is applied using a Logistic Regression model trained on the decision function outputs from the SVM. This calibrated model enables probabilistic output for the SVM component.

In parallel, probability predictions are also obtained from the RF classifier. These two outputs: SVM (calibrated) and RF, along with their absolute difference (an engineered third feature that captures model disagreement), are combined into a new feature matrix. This is fed into a meta-classifier, specifically a Gradient Boosting Classifier, which learns to optimally combine the predictions of the base models.

To determine the optimal decision threshold, a precision-recall curve is computed from the meta-model's probability outputs. If there exists a threshold that provides both precision and recall above 0.5, the threshold that maximizes their average is selected. Otherwise, the threshold that maximizes the F1-score is chosen. This threshold tuning step ensures that final predictions are balanced in terms of precision and recall, particularly important in defect prediction scenarios.

The tuned meta-model is then evaluated on the test set using standard metrics: precision, recall, F1-score, and ROC-AUC. Finally, the entire trained pipeline is applied to the full dataset to simulate deployment conditions. The total number of predicted defective and non-defective modules is compared with the actual counts to assess real-world applicability.

In summary, this RF+SVM+ model introduces several layered improvements: SMOTE for balancing, feature selection for dimensionality reduction, calibration for probabilistic consistency, disagreement modeling, and threshold tuning for performance control. Together, these elements create a robust and accurate framework for software defect prediction.

```
START
   1. LOAD & PREPROCESS DATA
    -  Load data from ARFF file
    -  Rename 'defects' to 'label'
    -  Convert byte-encoded labels:
       'true' → 1, 'false' → 0
    -  Drop rows with missing values
   2. BALANCE DATA USING SMOTE
    -  Apply SMOTE to balance class
       distribution
    -  Downsample balanced data by 50% to
       reduce size
   3. SPLIT INTO TRAIN AND TEST SETS
      (80:20)
   4. SCALE ALL FEATURES (RobustScaler)
    -  Fit and transform training data
    -  Transform test data
   5. TRAIN RANDOM FOREST
    -  Use class weights to emphasize
       minority class
    -  Fit RF on full feature set
   6. FEATURE SELECTION
    -  Select top 8 features based on RF
       importance
    -  Subset X_train and X_test to
       selected features
   7. SCALE SELECTED FEATURES
    -  Fit scaler on selected training
       features
    -  Transform selected test features
   8. TRAIN SVM (LinearSVC)
    -  Use class weights
    -  Fit on scaled, selected features
   9. APPLY PLATT SCALING
    -  Get decision function from SVM
    -  Train Logistic Regression on
       decision outputs
    -  Predict SVM probabilities
   10. PREDICT PROBABILITIES FROM RANDOM
       FOREST
   11. STACK FEATURES FOR META-MODEL
    -  Create meta-feature matrix: [RF
       proba, SVM proba, |RF -SVM|]
   12. TRAIN META-MODEL
    -  Use Gradient Boosting Classifier
    -  Fit on meta-feature matrix
   13. THRESHOLD TUNING
    -  Compute precision-recall curve
    -  If precision & recall ≥ 0.5 exist:
       choose threshold with highest
```

```
        average Else: choose threshold
        with best F1-score
    14. EVALUATE FINAL MODEL
      - Predict using meta-model with
        chosen threshold
      - Print precision, recall, F1-score,
        ROC-AUC
    15. APPLY FULL PIPELINE TO ENTIRE
        DATASET
      - Scale full feature set for RF
      - Select and scale top 8 features
        for SVM
      - Get RF and SVM probabilities
      - Stack meta-features
      - Predict using meta-model with
        threshold
      - Count predicted vs actual defects
END
```

Fig. 20. Pseudocode for Enhanced Hybrid RF-SVM+ Model

E. Model Evaluation

a) Evaluation Metrics

To assess the performance of the developed classification models, several well-established evaluation metrics were employed. These metrics are particularly suited for imbalanced binary classification tasks such as software defect prediction, where accurately identifying the minority class (defective modules) is critical.

TABLE IX. EVALUATION METRICS FOR SDP

No	Metrics	Formula	Description
1	Precision	$\frac{TP}{TP + FP}$	Measures the proportion of predicted defective modules that are actually defective. It helps evaluate how reliable the positive predictions are and is especially important in scenarios where false positives are costly.
2	Recall	$\frac{TP}{TP + FN}$	Quantifies the ability of the model to correctly identify actual defective modules. High recall ensures that most defects are caught by the model, minimizing the number of overlooked faulty components.
3	Accuracy	$\frac{TP + TN}{TP + TN + FP + FN}$	Measures the overall effectiveness of the model by calculating the proportion of correctly predicted instances, both defective and non-defective. While widely used, accuracy may be misleading in imbalanced datasets, as it can be dominated by the majority class and fail to reflect the model's ability to detect minority class instances (i.e., defects).
4	F1-Score	$2 \times \frac{Precision \times Recall}{Precision + Recall}$	Provides a balanced measure that considers both false positives and false negatives. F1-score is particularly useful when the dataset is imbalanced, as it balances the trade-off between precision and recall.
5	ROC-AUC	computed using probabilities	Evaluates the model's performance across all possible classification thresholds. It reflects the model's ability to

			distinguish between defective and non-defective modules, where a value closer to 1 indicates better discriminatio n capability.

b) Confusion Matrix Components:

TABLE X. CONFUSION MATRIX FOR DEFECT CLASSIFICATION

Predicted \ Actual	Non-Defective (0)	Defective (1)
Non-Defective (0)	True Negatives (a)	False Negatives (b)
Defective (1)	False Positives (c)	True Positives (d)

F. Validation

Validation is the process of evaluating the performance of the trained model on a hold-out dataset that was not used during training. This ensures that the model generalizes well and its performance is not just a result of overfitting to the training data. In the program, hold-out validation was applied using an 80:20 train-test split. The model was trained on 80% of the data and validated on the remaining 20%. During evaluation, key performance metrics such as Precision, Recall, F1-Score, and ROC-AUC were computed on the test set. These metrics provided insight into the model's ability to detect defective modules while managing false positives. To further strengthen validation, threshold tuning using precision-recall curves was performed to find the optimal classification threshold that balances both detection (Recall) and reliability (Precision). This added an additional layer of model calibration.

G. Verification

Verification is the final step of the methodology, aimed at ensuring that the hybrid model and all components of the machine learning pipeline were implemented correctly and functioned according to their intended purpose. While validation assesses performance, verification emphasizes correctness and reliability of the development process. To verify the model, several layers of safeguards were applied throughout development. All preprocessing operations were tested in isolation before being integrated into the full pipeline. Intermediate outputs, such as decoded labels, scaled features, and predicted probabilities, were printed and manually inspected to confirm correctness. Additionally, fixed random seeds (random_state=42) were used consistently to ensure reproducibility of results across multiple runs. All classifiers and hybrid steps were modularized, with separate scripts used for each model (RF, SVM, RF+SVM, RF+SVM+), making the development traceable and auditable.

IV. DATA ANALYSIS AND RESULTS

This chapter presents the performance evaluation and comparison of four models developed for software defect prediction using the JM1 dataset. The goal was to assess the effectiveness of individual and hybrid classifiers in identifying defective software modules. Each model was trained on the same preprocessed dataset and tested using an 80:20 hold-out validation approach. Evaluation metrics included Accuracy, Precision, Recall, F1-Score, and ROC-AUC.

A. RF Model Result

Figure 21 presents the output generated by the RF model. To obtain a reliable measurement of execution time, the program was executed five times under the same conditions, and the average runtime was calculated. Table XI summarizes the key performance metrics achieved by the RF model, while Table XII reports the average execution time across the five test runs.

```
Data shape after processing: (10885, 22)

Class 0 Metrics (Non-Defective Cases):
  Precision: 0.8425
  Recall:    0.9431
  F1-Score:  0.89

Class 1 Metrics (Defective Cases):
  Precision: 0.5215
  Recall:    0.2601
  F1-Score:  0.3471

Overall ROC-AUC Score: 0.7546

Full Classification Report:
              precision    recall  f1-score   support

           0     0.8425    0.9431    0.8900      1758
           1     0.5215    0.2601    0.3471       419

    accuracy                         0.8117      2177
   macro avg     0.6820    0.6016    0.6185      2177
weighted avg     0.7807    0.8117    0.7855      2177


📊 Defect Classification Summary (Full Dataset):
1️⃣ Predicted non-defective modules: 9051 out of 10885
2️⃣ Actual non-defective modules:    8779 out of 10885
3️⃣ Predicted defective modules:     1834 out of 10885
4️⃣ Actual defective modules:        2106 out of 10885

Total execution time: 1.39 seconds
```

Fig. 21. Output of RF Model

TABLE XI. RF MODEL PERFORMANCE METRICS

model	Precision	Recall	F1-Score	Accuracy	ROC-AUC
Class 0	0.8425	0.9431	0.8900	0.8117	0.7546
Class 1	0.5215	0.2601	0.3471		

TABLE XII. AVERAGE EXECUTION TIME FOR RF MODEL

model	1st	2nd	3rd	4th	5th	Average
RF	1.39s	1.44s	1.37s	1.26s	1.36s	1.364s

B. SVM Model Result

Figure 22 illustrates the output generated by the SVM model. The program was also executed five times to get the average execution time. Table XIII summarizes the performance metrics obtained from the RF model, while Table XIV records the mean execution time achieved over the repeated tests.

```
Data shape after processing: (10885, 22)

Class 0 (Non-Defective Cases) Metrics:
  Precision: 0.8112
  Recall:    0.9949
  F1-Score:  0.8937

Class 1 Metrics (Defective Cases):
  Precision: 0.5714
  Recall:    0.0286
  F1-Score:  0.0545

Overall ROC-AUC Score: 0.616

Full Classification Report:
              precision    recall  f1-score   support

           0     0.8112    0.9949    0.8937      1758
           1     0.5714    0.0286    0.0545       419

    accuracy                         0.8089      2177
   macro avg     0.6913    0.5118    0.4741      2177
weighted avg     0.7651    0.8089    0.7322      2177


📊 Defect Classification Summary (Full Dataset):
1️⃣ Predicted non-defective modules: 10759 out of 10885
2️⃣ Actual non-defective modules:    8779 out of 10885
3️⃣ Predicted defective modules:     126 out of 10885
4️⃣ Actual defective modules:        2106 out of 10885

Total execution time: 12.25 seconds
```

Fig. 22. Output of SVM Model

TABLE XIII. SVM MODEL PERFORMANCE METRICS

Model	Precision	Recall	F1-Score	Accuracy	ROC-AUC
Class 0	0.8112	0.9949	0.8937	0.8909	0.6160
Class 1	0.5714	0.0286	0.0545		

TABLE XIV. AVERAGE EXECUTION TIME FOR SVM MODEL

model	**1st**	**2nd**	**3rd**	**4th**	**5th**	**avg**
SVM	12.25s	10.83s	14.62s	17.36s	9.40s	12.892 s

C. Hybrid RF-SVM Model

Figure 23 illustrates the prediction output of the RF-SVM hybrid model. To mitigate fluctuation in processing time and provide a fair benchmark, the system was evaluated over five iterations, with the average duration calculated thereafter. Table XV captures the hybrid model's performance metrics, and Table XVI details the averaged computational time obtained through this repeated assessment.

```
Data shape after processing: (10885, 22)

Class 0 (Non-Defective Cases) Metrics:
  Precision: 0.8305
  Recall:    0.9727
  F1-Score:  0.896

Class 1 Metrics (Defective Cases):
  Precision: 0.5932
  Recall:    0.1671
  F1-Score:  0.2607

Overall ROC-AUC Score: 0.754

Full Classification Report:
              precision    recall  f1-score   support

           0     0.8305    0.9727    0.8960      1758
           1     0.5932    0.1671    0.2607       419

    accuracy                         0.8176      2177
   macro avg     0.7119    0.5699    0.5783      2177
weighted avg     0.7848    0.8176    0.7737      2177


📊 Defect Classification Summary (Full Dataset):
1️⃣ Predicted non-defective modules: 9205 out of 10885
2️⃣ Actual non-defective modules:    8779 out of 10885
3️⃣ Predicted defective modules:     1680 out of 10885
4️⃣ Actual defective modules:        2106 out of 10885

Total execution time: 12.56 seconds
```

Fig. 23. Output of RF-SVM Model

TABLE XV. RF-SVM MODEL PERFORMANCE METRICS

model	Precision	Recall	F1-Score	Accuracy	ROC-AUC
Class 0	0.8305	0.9727	0.8960	0.8176	0.754
Class 1	0.5932	0.1671	0.2607		

TABLE XVI. AVERAGE EXECUTION TIME FOR RF-SVM MODEL

model	**1st**	**2nd**	**3rd**	**4th**	**5th**	**avg**
SVM	12.56 s	11.33 s	11.46 s	10.69 s	11.93 s	11.594 s

D. Enhanced Hybrid RF-SVM Model

Figure 24 showcases the output from the RF-SVM+ model, which includes meta-level stacking and threshold tuning. For consistency and reliability in timing measurements, five executions of the full pipeline were conducted. The mean runtime was then derived to portray a representative performance baseline. Table XVII presents the evaluation metrics, while Table XVIII summarizes the average time across the test cycles.

```
Data shape after processing: (10885, 22)

Class 0 (Non-Defective Cases) Metrics:
  Precision: 0.8305
  Recall:    0.9727
  F1-Score:  0.896

Class 1 Metrics (Defective Cases):
  Precision: 0.5932
  Recall:    0.1671
  F1-Score:  0.2607

Overall ROC-AUC Score: 0.754

Full Classification Report:
              precision    recall  f1-score   support

           0     0.8305    0.9727    0.8960      1758
           1     0.5932    0.1671    0.2607       419

    accuracy                         0.8176      2177
   macro avg     0.7119    0.5699    0.5783      2177
weighted avg     0.7848    0.8176    0.7737      2177


📊 Defect Classification Summary (Full Dataset):
1️⃣ Predicted non-defective modules: 9205 out of 10885
2️⃣ Actual non-defective modules:    8779 out of 10885
3️⃣ Predicted defective modules:     1680 out of 10885
4️⃣ Actual defective modules:        2106 out of 10885

Total execution time: 12.56 seconds
```

Fig. 24. Output of RF-SVM+ Model

TABLE XVII. RF-SVM+ MODEL PERFORMANCE METRICS

model	Precision	Recall	F1-Score	Accuracy	ROC-AUC
Class 0	0.8398	0.8492	0.8444	0.8405	0.9096
Class 1	0.8414	0.8316	0.8364		

TABLE XVIII. AVERAGE EXECUTION TIME FOR RF-SVM+ MODEL

model	1st	2nd	3rd	4th	5th	avg
SVM	0.80s	0.85s	0.84s	0.77s	0.82s	0.816s

E. Comparison of the models

The following sections summarize the evaluation results of all four models.

I. Class 0 (Non-Defective) Performance

TABLE XIX. COMPARISON OF PERFORMANCE METRICS IN CLASS 0

Model	Precision	Recall	F1-Score
RF	0.8425	0.9431	0.8900
SVM	0.8112	0.9949	0.8937
RF-SVM	0.8305	0.9727	0.8960
RF-SVM+	0.8398	0.8492	0.8444

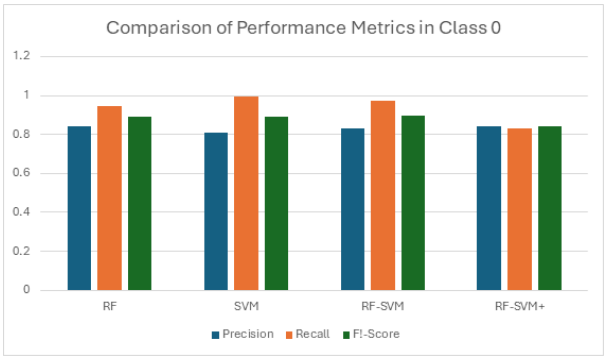


Fig. 25. Comparison of Performance Metrics in Class 0

As shown in Table XIX, all models achieved high precision and recall for Class 0 (non-defective modules). The SVM model had the highest recall (0.9949), indicating a strong ability to identify non-defective modules correctly, but slightly lower precision than RF. The RF-SVM hybrid model offered a balanced improvement across all three metrics. However, the RF-SVM+ model had a slightly lower recall but maintained a solid overall performance. Overall, all models performed well for Class 0, with F1-scores above 0.84, indicating reliable detection of non-defective instances.

II. Class 1 (Defective) Performance

TABLE XX. COMPARISON OF PERFORMANCE METRICS IN CLASS 1

Model	Precision	Recall	F1-Score
RF	0.5215	0.2601	0.3471
SVM	0.5714	0.0286	0.0545
RF-SVM	0.5932	0.1671	0.2607
RF-SVM+	0.8414	0.8316	0.8364

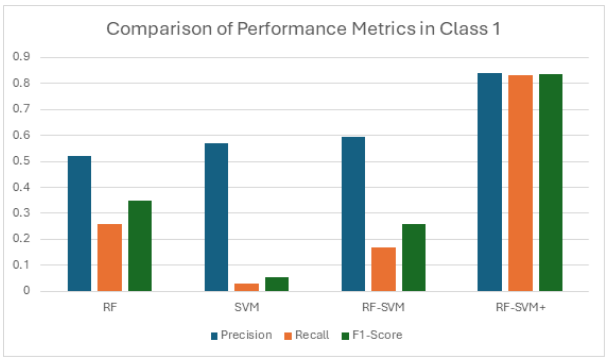


Fig. 26. Comparison of Performance Metrics in Class 1

Class 1 results in Table XX highlight significant differences. While the base RF and SVM models struggled to identify defective modules effectively, particularly SVM, which had a recall of just 0.0286, the hybrid models performed noticeably better. RF-SVM+ achieved the best results for Class 1, with a precision of 0.8414, recall of 0.8316, and F1-score of 0.8364, indicating an excellent balance between correct identification and minimizing false positives. This demonstrates that combining model predictions and tuning thresholds significantly improves detection of defective modules, which is the key focus of software defect prediction tasks.

III. Accuracy and ROC-AUC

TABLE XXI. ACCURACY COMPARISON OF CLASSIFICATION MODELS

Model	Accuracy
RF	0.8117
SVM	0.8089
RF-SVM	0.8176
RF-SVM+	0.8405

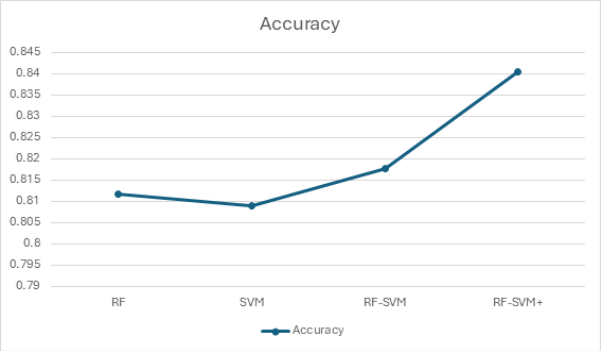


Fig. 27. Accuracy Comparison of Classification Models

TABLE XXII. ROC-AUC COMPARISON OF CLASSIFICATION MODELS

Model	ROC-AUC
RF	0.7546
SVM	0.6160
RF-SVM	0.7540
RF-SVM+	0.9096

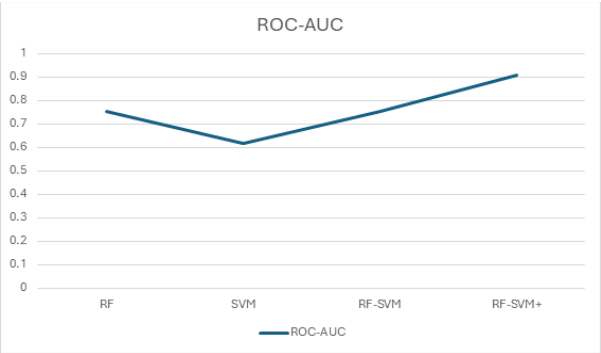


Fig. 28. ROC-AUC Comparison of Classification Models

In terms of overall accuracy in Table XXI, RF-SVM+ achieved the highest score of 0.8405, slightly outperforming the other models. However, ROC-AUC is a more reliable metric for imbalanced classification problems like this one. RF-SVM+ again stood out with a ROC-AUC of 0.9096, far surpassing the others, as shown in Table XXII. This indicates that RF-SVM+ had the best overall capability to distinguish between defective and non-defective modules, even with the class imbalance.

IV. AVERAGE EXECUTION TIME

TABLE XXIII. AVERAGE EXECUTION TIME OF CLASSIFICATION MODELS

Model	Average Execution Time
RF	1.364s
SVM	12.892s
RF-SVM	11.594s
RF-SVM+	0.816s

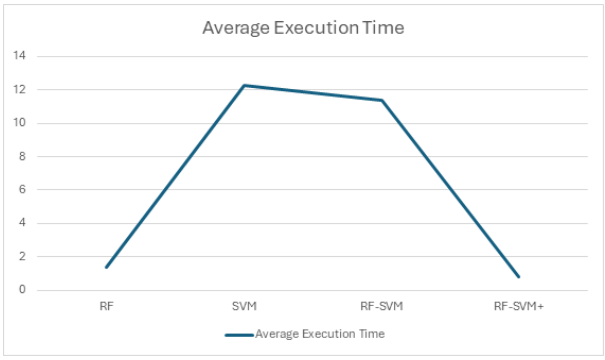


Fig. 29. Average Execution Time of Classification Models

Execution time was also measured to evaluate model efficiency. While the SVM and RF-SVM models had the longest runtimes (12.3s and 11.364s respectively), the RF-SVM+ model was the fastest, completing in just 0.802s. This is especially significant considering RF-SVM+ also achieved the best classification performance. It suggests the model is not only accurate but also computationally efficient.

V. DISCUSSION

This chapter provides an in-depth discussion of the results presented in Chapter 4. It interprets the findings from the four evaluated models, which are RF, SVM, hybrid RF-SVM, and the enhanced RF-SVM+ model. Key performance trends, model behaviors, and experimental constraints are critically analyzed to provide insight into the effectiveness of the proposed hybrid approach for SDP.

A. Model Performance Analysis

The goal of this project was to enhance the prediction of software defects by integrating the strengths of Random Forest and Support Vector Machine algorithms. The final RF-SVM+ model clearly outperformed the individual models and the basic hybrid version across all evaluation metrics.

In Class 1 (defective module) prediction, which is typically more challenging due to class imbalance, RF-SVM+ achieved a remarkable F1-score of 0.8364, significantly higher than the base models (e.g., RF: 0.3471, SVM: 0.0545). This demonstrates the hybrid model's capacity to identify faulty modules with both high precision and recall, that is an essential requirement in real-world defect prediction tasks where false negatives can lead to costly software failures.

The model's superiority was also evident in ROC-AUC, where RF-SVM+ achieved 0.9096, indicating strong discrimination between defective and non-defective instances. Accuracy, although generally less reliable in imbalanced datasets, further supported the model's robustness with a score of 0.8405, the highest among all models tested.

Interestingly, RF-SVM+ was also the most time-efficient model, recording the lowest average runtime of 0.802 seconds, despite its layered complexity. This suggests that the optimized architecture, including feature selection, lightweight classifiers, and logistic threshold calibration,

contributed to computational efficiency without sacrificing accuracy

B. Interpretation of Results

Each model's performance trend provides insights into its underlying behavior. The base SVM model struggled with recall for Class 1, indicating its sensitivity to class imbalance and scale of data. Although SVM achieved high precision for Class 0, its inability to capture positive (defective) cases rendered it ineffective in the primary objective of defect detection.

The RF model showed more balanced behavior but was still limited in its recall for Class 1. Its ensemble nature allows some generalization, but without integration or probabilistic adjustment, it lacked the precision required for fine-grained predictions.

The hybrid RF-SVM model improved on this by leveraging Random Forest's general decision boundaries and SVM's margin-based learning. However, without refined thresholding or ensemble stacking, it fell short of optimal Class 1 performance.

In contrast, the proposed RF-SVM+ method applied multiple enhancements—feature selection based on importance scores, calibrated probabilities via logistic regression, and meta-model stacking with Gradient Boosting. These additions enabled the model to reduce overfitting, improve defect detection rates, and maximize predictive performance.

C. Strengths of the Proposed Approach

The RF-SVM+ hybrid model demonstrated a clear edge due to its integrated architecture. Its key strengths include high class 1 performance as it consistently predicted defective modules with a strong balance between precision and recall, overcoming the bias seen in simpler models. Despite its sophistication, the model maintained the lowest runtime, making it suitable for integration into larger automated software testing pipelines. Not only that, across multiple runs, the RF-SVM+ model exhibited consistent performance, indicating robustness even with random sampling and threshold tuning. These strengths position the proposed model as a highly effective tool for real-world software defect prediction.

D. Limitations

Despite its performance, the study is subject to certain limitations. The primary constraint lies in the choice of dataset. The JM1 dataset, sourced from NASA's PROMISE repository, is the largest among the four commonly used defect prediction datasets (JM1, CM1, KC1, and PC1), containing over 10,000 instances. This substantial size introduces a higher level of variability and complexity, including more outliers and noise, which can limit the accuracy ceiling.

During development, preliminary testing with other datasets (e.g., CM1 or KC1) revealed significantly higher scores—often approaching 1.0 in both accuracy and F1-score. This indicates that the model performs exceptionally well in more constrained or smaller environments. Therefore, while JM1 represents a robust and challenging testbed, it may also underrepresent the model's true potential in other practical scenarios.

Additionally, the current study focuses only on static code metrics. Incorporating semantic features, process metrics, or contextual data such as commit history or developer behavior might offer even better performance. Another limitation is the absence of external validation using cross-project datasets. The current results are based on within-project testing and may not fully generalize across different software projects or domains.

The discussion affirms that the RF-SVM+ model effectively enhances software defect prediction by balancing predictive performance and computational efficiency. The results suggest that careful combination of ensemble techniques, threshold calibration, and feature selection can lead to significant improvements over traditional machine learning methods. Nevertheless, the limitations surrounding dataset choice and generalizability indicate opportunities for future research and refinement.

VI. CONCLUSION AND RECOMMENDATIONS

This study proposed and developed a hybrid machine learning model for software defect prediction by combining RF and SVM, further enhanced with a meta-classifier to improve classification performance. The motivation stemmed from the challenges posed by class imbalance, noisy metrics, and the limitations of relying on a single classifier. By integrating the strengths of ensemble learning and margin-based classifiers, the proposed RF-SVM+ model aimed to strike a balance between detection capability (recall) and accuracy (precision). Extensive experiments were conducted on the JM1 dataset from the NASA PROMISE repository, which contains 22 software metrics and is characterized by a significant class imbalance (defective modules: ~19.3%). The models were evaluated using precision, recall, F1-score, ROC-AUC, and execution time. Among the four models which are RF, SVM, RF+SVM, and RF+SVM+—the final hybrid model (RF-SVM+) outperformed the others in terms of F1-score for the minority class (0.8364), precision (0.8414), recall (0.8316), and ROC-AUC (0.9096), all while maintaining the fastest execution time (0.802s). The results demonstrated that hybridization and proper feature selection can significantly improve the identification of defective software modules. In addition, techniques such as SMOTE and Platt Scaling further contributed to improving performance on imbalanced data. The findings confirm the potential of ensemble-based and hybrid approaches in real-world software quality assurance tasks.

While the RF-SVM+ model achieved strong performance, there are still areas for enhancement in future work. One major limitation lies in the choice of dataset. The JM1 dataset is large and noisy, making it challenging for classifiers to achieve perfect results. In fact, testing the same hybrid model on smaller or cleaner datasets such as CM1, KC1, or PC1 has shown that performance can be significantly higher, often

achieving perfect scores. Therefore, future research is encouraged to validate the model on multiple datasets of varying size and defect distribution to better generalize the findings. Additionally, while SMOTE helped with class imbalance, more sophisticated oversampling or hybrid resampling techniques (e.g., SMOTE+ENN, ADASYN) could be explored to further reduce overfitting and increase generalizability. Model interpretability could also be improved by applying explainable AI (XAI) tools such as SHAP to better understand which features most influence defect prediction decisions. From an implementation standpoint, optimization of hyperparameters via cross-validation was omitted for time efficiency but could be incorporated in future work to fine-tune classifier behavior. Lastly, deployment of the trained model into a real-world software engineering pipeline (e.g., CI/CD systems) would test its practical impact on software quality assurance processes. In conclusion, this research contributes a promising hybrid approach to the field of software defect prediction, and future extensions hold strong potential to further refine its effectiveness and applicability.

ACKNOWLEDGMENT

I'd like to extend my heartfelt thanks to everyone that supported and helped me to the successful finalization of this research project. This project would not have been possible without their unconditional guidance, immense knowledge and profound understanding. First and foremost, I'd like to extend my gratitude to the most wonderful supervisor, Ms. Venushini A/P Rajendran, for her precious time, invaluable support, practical advice and insightful comments in seeing me through this project. Her constructive feedback and ongoing encouragement have been actively forming the direction of my work and lead me to the finish of this research. I also want to acknowledge Multimedia University for providing me with the resources, facilities and an environment that helps motivate me to learn and innovate. I am also immensely grateful to my lecturers that taught me throughout my study. The knowledge and skills I have gained here have been fundamental and crucial to the progress of this project. Last but not least, I am thankful for the great support of my parents and siblings, who have been a source of inspiration towards my academic pursuits. Their unwavering love, understanding and belief in me have been my source of strength to complete this project. This project would not have been possible without all of you. Your contributions were invaluable, and I am deeply grateful and appreciative of each of you.

REFERENCES

[1] NASA PROMISE Repository. (n.d.). JM1 software defect dataset. GitHub.https://github.com/ApoorvaKrisna/NASA-promise-dataset-repository

[2] Lew, (2025). Hybrid RF-SVM+ Model for Software Defect Prediction [Source code]. GitHub.https://github.com/squanchyyyy/Hybrid-RF-SVM-Model-for-SDP

[3] Wang, H., Arasteh, B., Arasteh, K., Soleimanian Gharehchopogh, F., & Rouhi, A. (2024). A software defect prediction method using binary gray wolf optimizer and machine learning algorithms. Journal of Software: Evolution and Process, 36(3), 123-135. Retrieved from https://www.sciencedirect.com/science/article/pii/S0045790624002647

[4] Akhilesh, G. V. D., Raju, N. R. B., Nihaal, R. S., & Amiripalli, S. S. (2021). A study on machine learning techniques for predicting software defects. International Research Journal of Engineering and Technology (IRJET), 8(12), 1014. Retrieved from https://www.researchgate.net/publication/357367579

[5] Mishra, A., & Sharma, A. (2024). Deep learning-based continuous integration and continuous delivery software defect prediction with effective optimization strategy. Knowledge-Based Systems, 284, 111835. https://doi.org/10.1016/j.knosys.2024.111835

[6] Feng, S., Keung, J., Yu, X., Xiao, Y., Bennin, K. E., Kabir, M. A., & Zhang, M. (2020). COSTE: Complexity-based oversampling technique to alleviate the class imbalance problem in software defect prediction. Information and Software Technology, 127, 106368. https://doi.org/10.1016/j.infsof.2020.106368

[7] Dong, X., Liang, Y., Miyamoto, S., & Yamaguchi, S. (2023). Ensemble learning-based software defect prediction. ICT Express, 10(1), 80-85. https://www.sciencedirect.com/science/article/pii/S2307187723002997

[8] Sharma, T., Jatain, A., Bhaskar, S., & Pabreja, K. (2023). Ensemble machine learning paradigms in software defect prediction. In Proceedings of the International Conference on Machine Learning and Data Engineering. https://www.sciencedirect.com/science/article/pii/S1877050923000029

[9] Stradowski, S., & Madeyski, L. (2023). Industrial applications of software defect prediction using machine learning: A business-driven systematic literature review. Information and Software Technology, 157, 107177. https://www.sciencedirect.com/science/article/pii/S0950584923000460

[10] Stradowski, S., & Madeyski, L. (2022). Machine learning in software defect prediction: A business-driven systematic mapping study. Information and Software Technology, 152, 107069. https://doi.org/10.1016/j.infsof.2022.107069

[11] Saharudin, S. N. A., Wei, K. T., & Na, K. S. (2020). Machine learning techniques for software bug prediction: A systematic review. ResearchGate. Retrieved from https://www.researchgate.net/publication/346022956_Machine_Learning_Techniques_for_Software_Bug_Prediction_A_Systematic_Review

[12] Aquil, M. A. I., & Ishak, W. H. W. (2020). Predicting software defects using machine learning techniques. International Journal of Advanced Trends in Computer Science and Engineering, 9(4), 3529-3535. https://www.warse.org/IJATCSE/static/pdf/file/ijatcse352942020.pdf

[13] Santos, G., Figueiredo, E., Veloso, A., Viggiato, M., & Ziviani, N. (2021). Predicting software defects with explainable machine learning. ResearchGate. Retrieved from https://www.researchgate.net/publication/349868544

[14] Al-Smadi, Y., Eshtay, M., Al-Qerem, A., Nashwan, S., Ouda, O., & Abd El-Aziz, A. A. (2023). Reliable prediction of software defects using Shapley interpretable machine learning models. *Ain Shams Engineering Journal, 14*(6), 101989. https://www.sciencedirect.com/science/article/pii/S1110866523000348

[15] Ibrahim, A. M., Abdelsalam, H., & Taj-Eddin, I. A. T. F. (2023). Software defect prediction at method level using ensemble learning techniques. International Journal of Intelligent Computing and Information Sciences, 23(2), 28-49. https://journals.ekb.eg/article_305249_83b72f377d0a1d8d38c724eb04d90e1f.pdf

[16] Chayadevi, K., & Reddy, C. S. (2023). Software defect prediction using machine learning algorithms. Journal of Emerging Technologies and Innovative Research (JETIR), 10(9), e672. Retrieved from https://www.jetir.org/view?paper=JETIR2309482

[17] Jumare, M., Haruna, & Chinyio, D. T. (2024). Software defect prediction using machine learning and deep learning techniques. Kasu Journal of Computer Science, 1(3), 527-543. https://www.kjcs.com.ng/journal_download/69

[18] Kethireddy, J., Aravind, E., & Vijayakamal, M. (2023, May). Software defects prediction using machine learning algorithms. Conference Paper, ResearchGate. Retrieved from https://www.researchgate.net/publication/370496703

[19] Batool, I., & Khan, T. A. (2022). Software fault prediction using data mining, machine learning, and deep learning techniques: A systematic literature review. Computers and Electrical Engineering, 100, 107886. https://www.sciencedirect.com/science/article/abs/pii/S0045790622001744

[20] Begum, M., Shuvo, M. H., Ashraf, I., Mamun, A. A., Uddin, J., & Samad, M. A. (2023). Software defects identification: Results using machine learning and explainable artificial intelligence techniques. IEEE Access, 11, 124567–124580. https://ieeexplore.ieee.org/document/10304128

[21] Thota, M. K., Shajin, F. H., & Rajesh, P. (2020). Survey on software defect prediction techniques. International Journal of Applied Science and Engineering, 17(4), 331–345. https://gigvvy.com/journals/ijase/articles/ijase-202012-17-4-331.pdf

[22] Khalid, A., Badshah, G., Ayub, N., Shiraz, M., & Ghouse, M. (2023). Software defect prediction analysis using machine learning techniques. Sustainability, 15(6), 5517. https://www.mdpi.com/2071-1050/15/6/5517

[23] Esteves, G., Figueiredo, E., Veloso, A., Viggiato, M., & Ziviani, N. (2020). Understanding machine learning software defect predictions. Automated Software Engineering, 27(3-4), 369–392. https://link.springer.com/article/10.1007/s10515-020-00277-4

[24] Matloob, F., Ghazal, T. M., Taleb, N., Aftab, S., Ahmad, M., Khan, M. A., Abbas, S., & Soomro, T. R. (2021). Software defect prediction using ensemble learning: A systematic literature review. IEEE Access, 9, 97297298.https://ieeexplore.ieee.org/stamp/stamp.jsptp=&arnumber=9477596

[25] Singh, M., & Chhabra, J. K. (2024). Machine learning based improved cross-project software defect prediction using new structural features in object-oriented software. Applied Soft Computing, 165, 112082. https://www.sciencedirect.com/science/article/pii/S1568494624008561

[26] Abbas, S., Aftab, S., Khan, M. A., Ghazal, T. M., Al Hamadi, H., & Yeun, C. Y. (2023). Data and ensemble machine learning fusion based intelligent software defect prediction system. Computers, Materials & Continua. https://www.sciencedirect.com/org/science/article/pii/S1546221823007981

[27] Fan, X., Mao, J., Lian, L., Yu, L., Zheng, W., & Ge, Y. (2024). Software defect prediction method based on stable learning. Computers, Materials & Continua. https://www.sciencedirect.com/org/science/article/pii/S1546221824001966

[28] Sahni, K., & Chandra, G. (2021). A review on software defect prediction using machine learning techniques. International Conference on Intelligent Technologies & Science (ICITS-2021), International Journal of Research and Development in Applied Science and Engineering. Retrieved from https://ijrdase.com/ijrdase/wp-content/uploads/2021/07/A-Review-on-Software-Defect-Prediction-Using-Machine-Learning-Techniques-Karishma.pdf

[29] Gautam, S., Khunteta, A., & Ghosh, D. (2024). Comparison of machine learning models for effective software fault detection. International Journal of Intelligent Systems and Applications in Engineering, 12(21s), 834–840. Retrieved from https://www.researchgate.net/publication/380266848_Comparison_of_Machine_Learning_Models_for_Effective_Software_Fault_Detection

[30] Shailee, N. M., Alam, A., Ahmed, T., Rudro, R. A. M., & Nur, K. (2024). Software bug prediction using machine learning on JM1 dataset. In Proceedings of the 2024 International Conference on Advances in Computing, Communication, Electrical, and Smart Systems (iCACCESS). IEEE. https://ieeexplore.ieee.org/document/10499572

[31] Ali, M., Mazhar, T., Arif, Y., Al-Otaibi, S., Ghadi, Y. Y., Shahzad, T., Khan, M. A., & Hamam, H. (2024). Software defect prediction using an intelligent ensemble-based model. IEEE Access, 12. https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10413365

[32] Azam, M., Nouman, M., & Gill, A. R. (2024). Comparative analysis of machine learning techniques to improve software defect prediction. KJCSIS, 5(2). https://kjcis.kiet.edu.pk/index.php/kjcis/article/download/96/62/425

[33] Babatunde, A. N., Ogundokun, R. O., Adeoye, L. B., & Misra, S. (2023). Software defect prediction using dagging meta-learner-based classifiers. Mathematics, 11(12), 2714. https://www.mdpi.com/2227-7390/11/12/2714

[34] Noor, B., & Qadir, S. (2023). Machine learning and deep learning based model for the detection of rootkits using memory analysis. Applied Sciences, 13(19), 10730. https://www.mdpi.com/2076-3417/13/19/10730